



计算机组成原理与接口技术（实验） ——基于MIPS架构

May, 2026

实验2&3 类MIPS单周期微处理器设计（8学时，第10-11周）

杨明
华中科技大学电子信息与通信学院
myang@hust.edu.cn



▶ 实验内容

- 目的
- 任务及时间安排
- 报告要求

▶ 原理回顾

- 计算机工作原理
- 简单微处理器的基本构成

▶ Vivado软件使用

- 模块设计
- 功能仿真

- ▶ 了解微处理器的基本结构
- ▶ 掌握哈佛结构的计算机工作原理
- ▶ 学会设计简单的微处理器
- ▶ 了解软件控制硬件工作的基本原理

► 任务

- **基本要求**：全部采用Verilog 硬件描述语言设计实现简单指令集MIPS 微处理器，要求指令存储器在时钟上升沿读出指令，指令指针的修改、寄存器文件写入、数据存储器数据写入都在时钟下降沿完成。完成完整设计代码输入、各模块完整功能仿真，整体仿真，验证所有指令执行情况。
- 且假定所有通用寄存器复位时取值都为各自寄存器编号乘以4；PC寄存器初始值为0；数据存储器 and 指令存储器容量大小为 32×32 ，且地址都从0开始，指令存储器初始化时装载测试MIPS汇编程序的机器指令，数据存储器所有存储单元的初始值为其对应地址的取值。需要注意的是数据存储器的地址呈现以下规则：都是4的整数倍。

- 仿真以下MIPS汇编语言程序段的执行流程：

```
main:  add    $4, $2, $3
      lw     $4, 4($2)
      sw     $5, 8($2)
      sub    $2, $4, $3
      or     $2, $4, $3
      and    $2, $4, $3
      slt    $2, $4, $3
      beq    $3, $3, equ
      lw     $2, 0($3)
equ:   beq    $3, $4, exit
      sw     $2, 0($3)
exit:  j      main
```

[提醒1] 在理解原理的基础上，自己想怎么编程就怎么编程。

[提醒2] 课本及课件步骤/代码不全，仅供参考。

另：各个小组所属成员需扩展实现各小组要求扩展的指令：

1) addi , ori ; 2) lb , lbu , lh , lhu ;

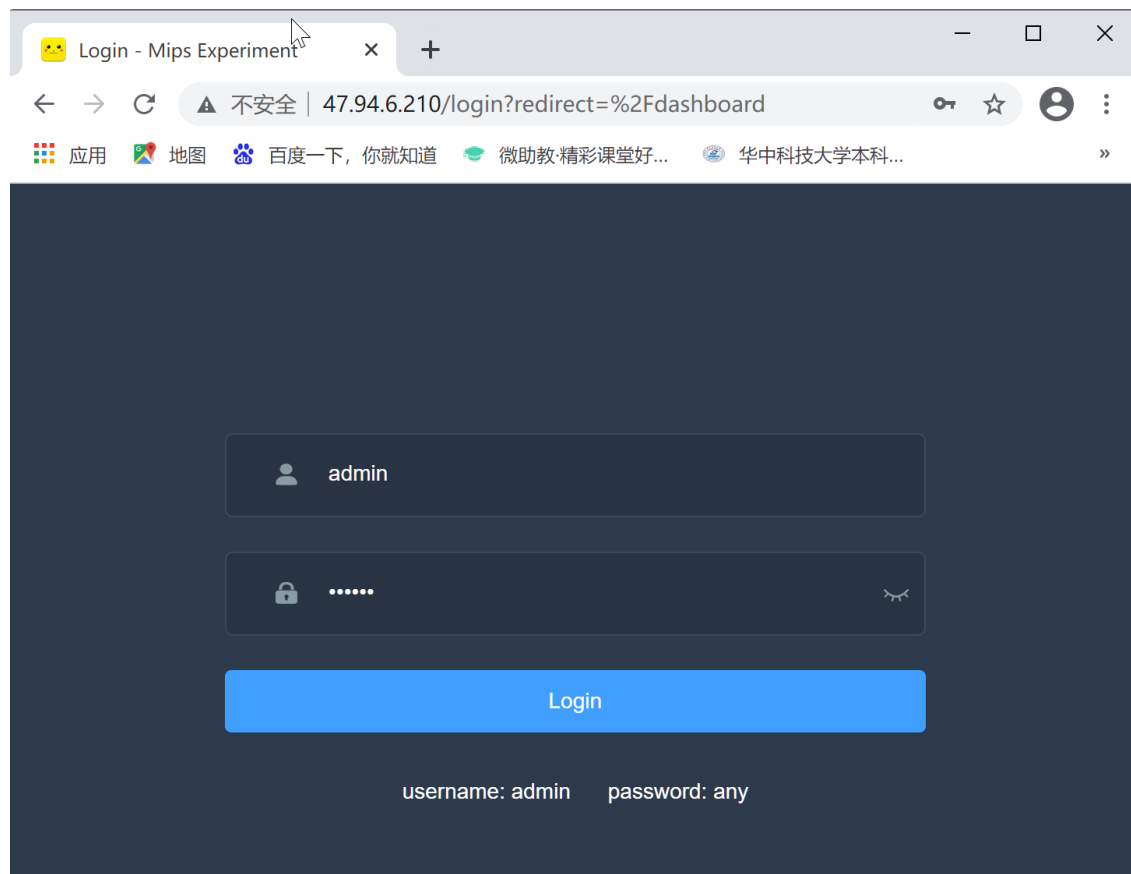
3) bne ; bltz ; bgez ; 4) jal , jr ;

并在汇编程序中添加相应指令仿真验证该指令执行是否正确。

- ▶ 1. 实验任务、目标
- ▶ 2. 微处理器各个模块硬件设计原理、verilog代码
- ▶ 3. Rom汇编程序设计、代码
- ▶ 4. 各个模块的仿真激励代码、仿真结果截图以及文字说明如何验证其正确性
- ▶ 5. 心得体会

► 在网站提交

- <http://47.94.6.210>



Agenda

► 实验内容

- 目的
- 任务及时间安排
- 报告要求

► 原理回顾

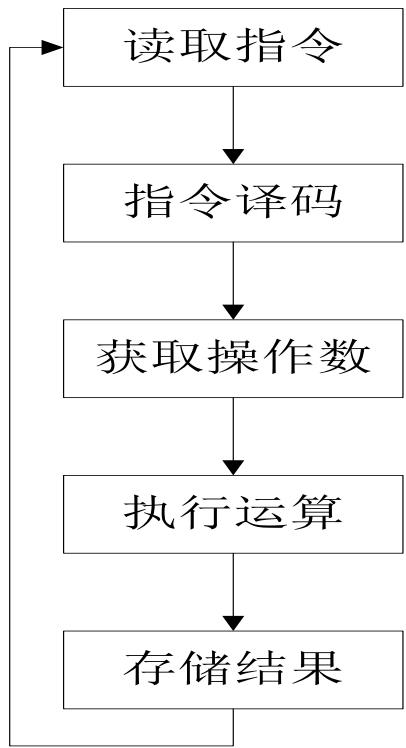
- 计算机工作原理
- 简单微处理器的基本构成

► Vivado软件使用

- 模块设计
- 功能仿真

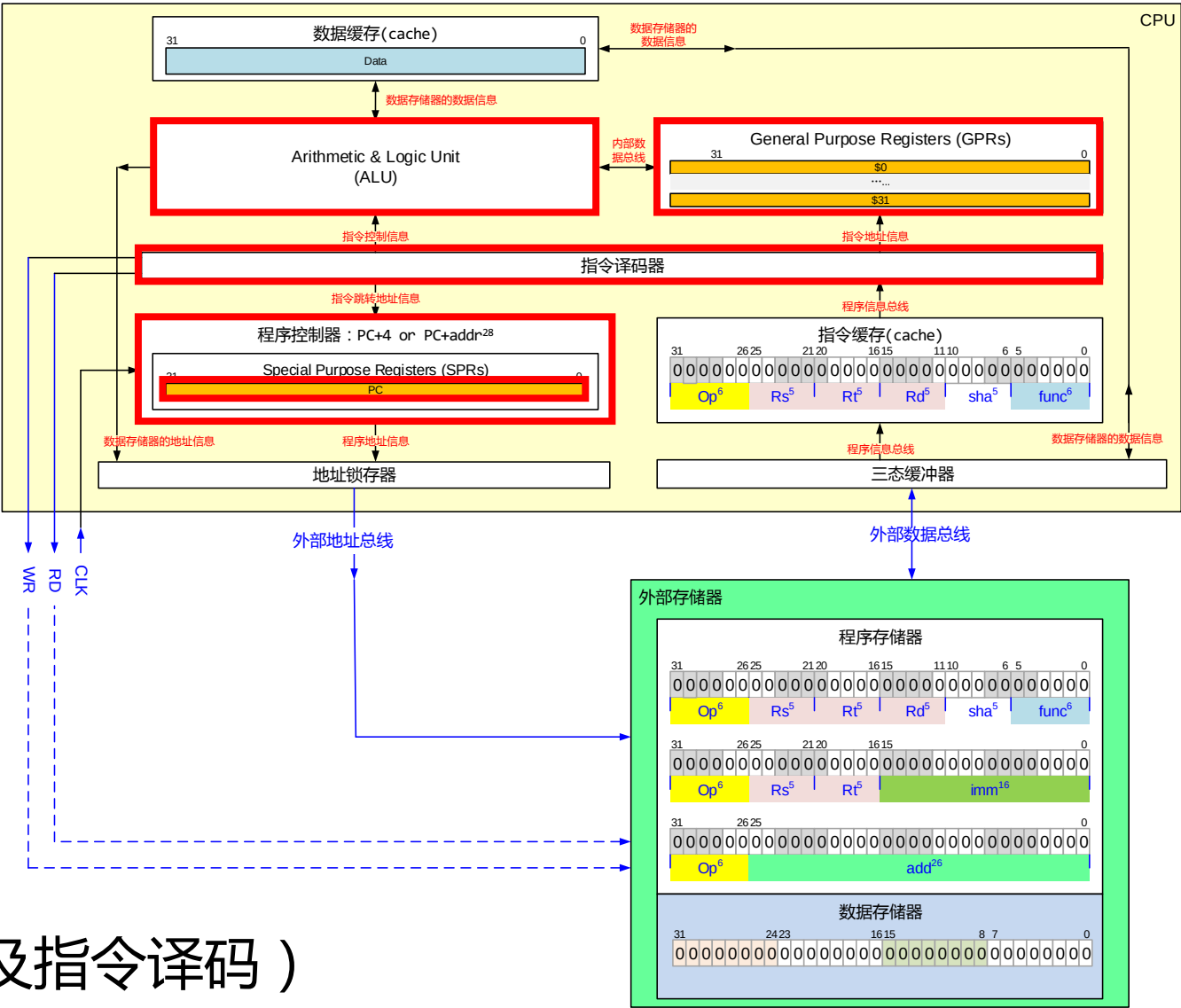
计算机工作原理

- 存储程序和程序控制”



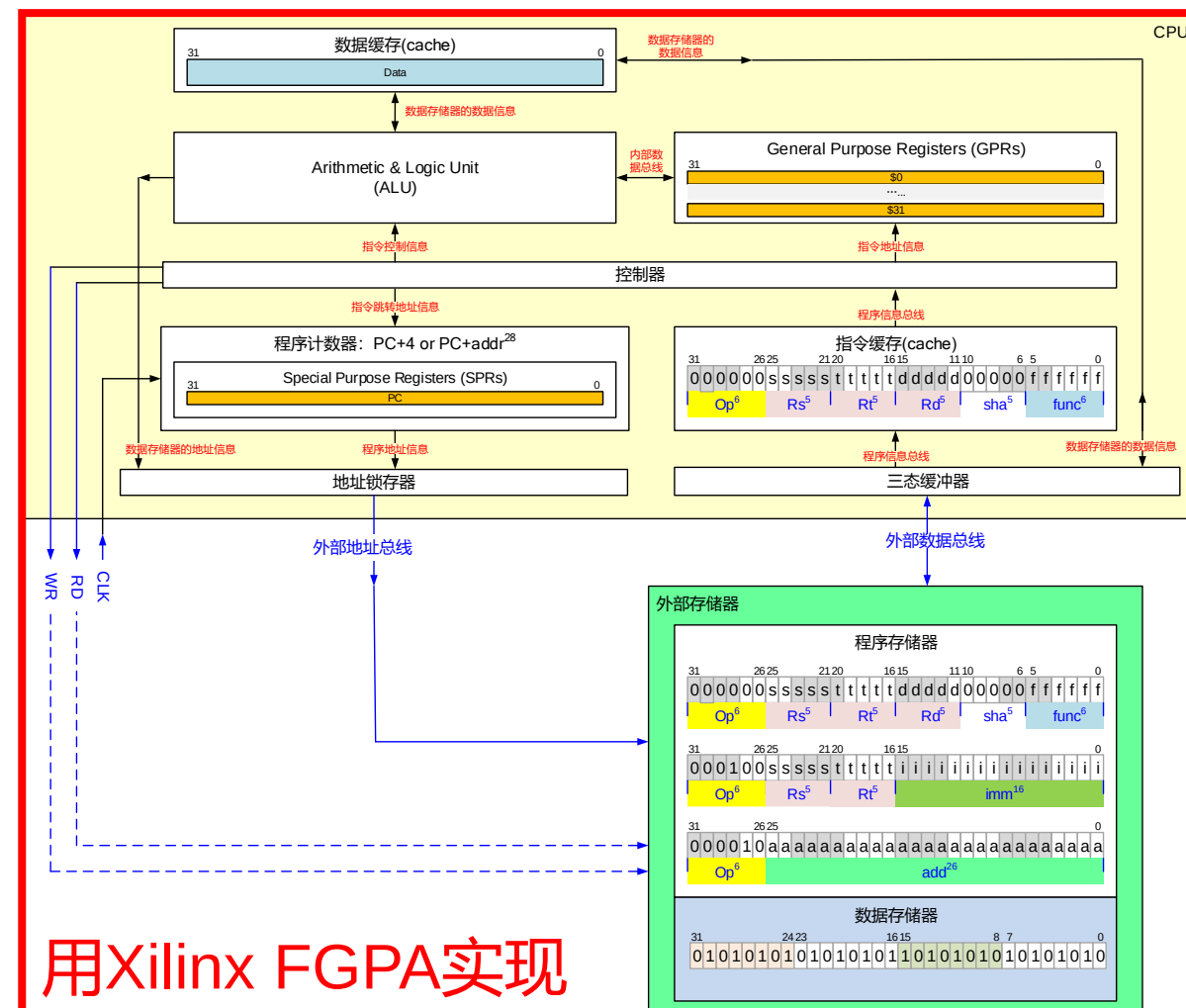
简单微处理器的基本构成

- 运算器、Regs、控制器（程序控制及指令译码）

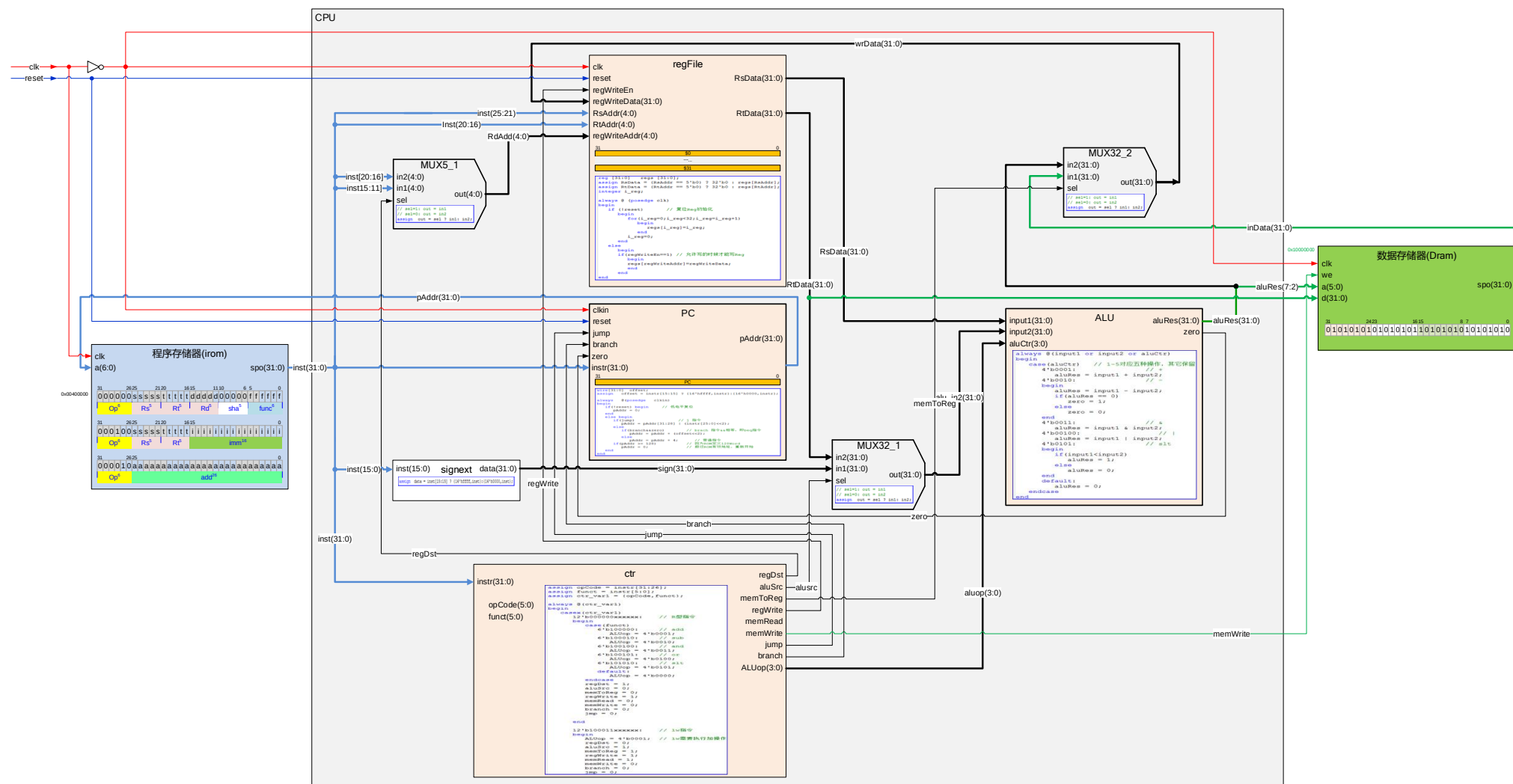


► 简单微处理器的基本构成

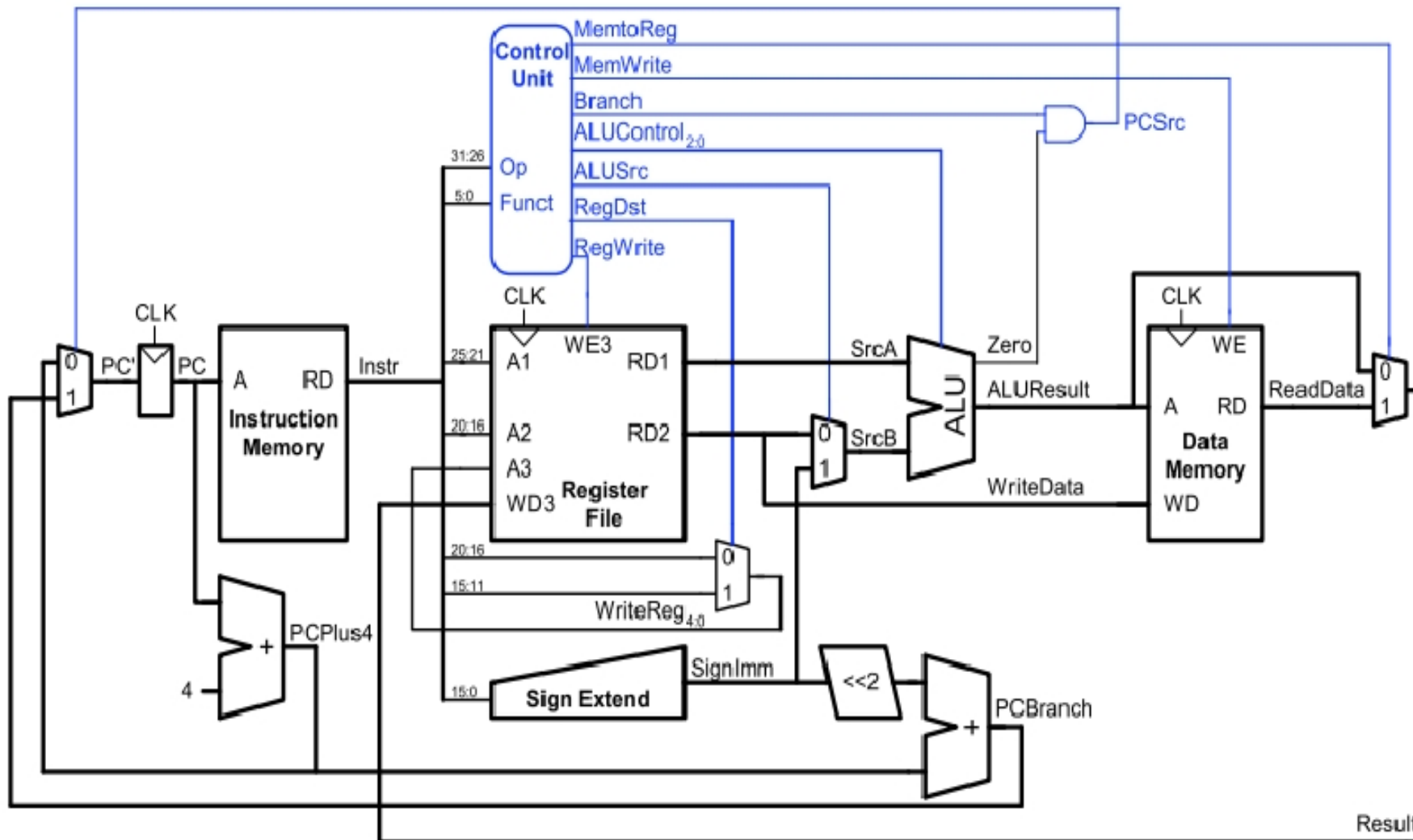
- 假设MIPS微处理器支持以下指令
 - 算术逻辑运算如add, sub, and, or, slt指令
 - 存储器读写: lw, sw指令
 - 程序控制如beq, j指令



► 简单微处理器的基本构成（哈佛结构）——框图1



► 简单微处理器的基本构成（哈佛结构）—— 框图2



指令类型及机器码

- 支持add, sub, and, or, slt, **andi**, lw, sw, beq, j 等指令
 - 加、减、与、或、slt的op-code均为000000, 通过function code来区分(2级译码):
 - Func = 100000 : add
 - Func = 100010 : sub
 - Func = 100100 : and
 - Func = 100101 : or
 - Func = 101010 : slt

只读指令

Add:
add Rd, Rs, Rt # RF[Rd] = RF[Rs] + RF[Rt]

Op-Code	Rs	Rt	Rd	Function Code
000000	sssss	ttttt	dddd	00000100000

Add contents of Reg.File[Rs] to Reg.File[Rt] and store result in Reg.File[Rd].
If overflow occurs in the two's complement number system, an exception is generated.

Subtract:
sub Rd, Rs, Rt # RF[Rd] = RF[Rs] - RF[Rt]

Op-Code	Rs	Rt	Rd	Function Code
000000	sssss	ttttt	dddd	00000100010

Subtract contents of Reg.File[Rt] from Reg.File[Rs] and store result in Reg.File[Rd].
If overflow occurs in the two's complement number system, an exception is generated.

And:
and Rd, Rs, Rt # RF[Rd] = RF[Rs] AND RF[Rt]

Op-Code	Rs	Rt	Rd	Function Code
000000	sssss	ttttt	dddd	00000100100

Bitwise logically AND contents of Register File[Rs] with Reg.File[Rt] and store result in Reg.File[Rd].

OR:
or Rd, Rs, Rt # RF[Rd] = RF[Rs] OR RF[Rt]

Op-Code	Rs	Rt	Rd	Function Code
000000	sssss	ttttt	dddd	00000100101

Bit wise logically OR contents of Register File[Rs] with Reg.File[Rt] and store result in Reg.File[Rd].

Set on Less Than: (Used in branch macro instructions)
slt Rd, Rs, Rt # if (RF[Rs] < RF[Rt]) then RF[Rd] = 1 else RF[Rd] = 0

Op-Code	Rs	Rt	Rd	Function Code
000000	sssss	ttttt	dddd	00000101010

If the contents of Reg.File[Rs] are less than the contents of Reg.File[Rt], then Reg.File[Rd] is set to one, otherwise Reg.File[Rd] is set to zero; assuming the two's complement number system representation.

一、二、三、四、五、六、七、八、九、十、十一、十二、十三、十四、十五、十六、十七、十八、十九、二十、二十一、二十二、二十三、二十四、二十五、二十六、二十七、二十八、二十九、三十、三十一、三十二、三十三、三十四、三十五、三十六、三十七、三十八、三十九、四十、四十一、四十二、四十三、四十四、四十五、四十六、四十七、四十八、四十九、五十、五十一、五十二、五十三、五十四、五十五、五十六、五十七、五十八、五十九、六十、六十一、六十二、六十三、六十四、六十五、六十六、六十七、六十八、六十九、七十、七十一、七十二、七十三、七十四、七十五、七十六、七十七、七十八、七十九、八十、八十一、八十二、八十三、八十四、八十五、八十六、八十七、八十八、八十九、九十、九十一、九十二、九十三、九十四、九十五、九十六、九十七、九十八、九十九、一百

Add Immediate:
addi Rt, Rs, Imm # RF[Rt] = RF[Rs] + Imm

Op-Code	Rs	Rt	Imm
001000	sssss	ttttt	iiiiiiiiiiiiiiii

Add contents of Reg.File[Rs] to sign extended Imm value, store result in Reg.File[Rt].
If overflow occurs in the two's complement number system, an exception is generated.

Load Word:
lw Rt, offset(Rs) # RF[Rt] = Mem[RF[Rs] + Offset]

Op-Code	Rs	Rt	Offset
100011	sssss	ttttt	iiiiiiiiiiiiiiii

The 16-bit offset is sign extended and added to Reg.File[Rs] to form an effective address. A 32-bit word is read from memory at the effective address and loaded into Reg.File[Rt].
If the least two significant bits of the effective address are not zero, an address error exception occurs. There are four bytes in a word, so word addresses must be binary numbers that are a multiple of four, otherwise an address error exception occurs.

Store Word:
sw Rt, offset(Rs) # Mem[RF[Rs] + Offset] = RF[Rt]

Op-Code	Rs	Rt	Offset
101011	sssss	ttttt	iiiiiiiiiiiiiiii

The 16-bit offset is sign extended and added to Reg.File[Rs] to form an effective address. The contents of Reg.File[Rt] are stored in memory at the effective address. If the least two significant bits of the effective address are not zero, an address error exception occurs. There are four bytes in a word, so word addresses must be binary numbers that are a multiple of four, otherwise an address error exception occurs.

Branch if Equal:
beq Rs, Rt, Label # If (RF[Rs] == RF[Rt]) then PC = PC + Imm << 2

Op-Code	Rs	Rt	Imm
000100	sssss	ttttt	iiiiiiiiiiiiiiii

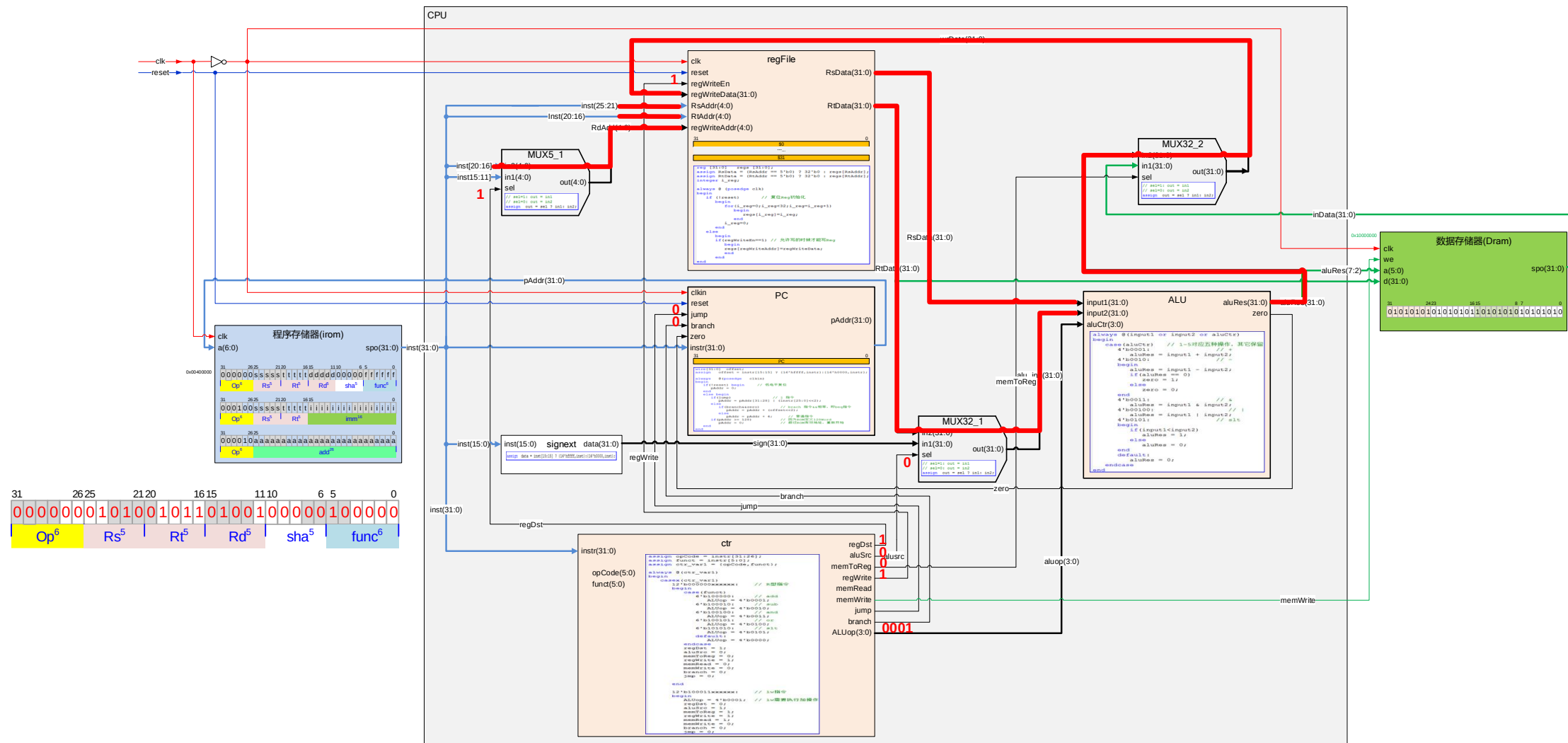
If Reg.File[Rs] is equal to Reg.File[Rt] then branch to label.

Jump:
j Label # PC = PC(31:28) | Imm << 2

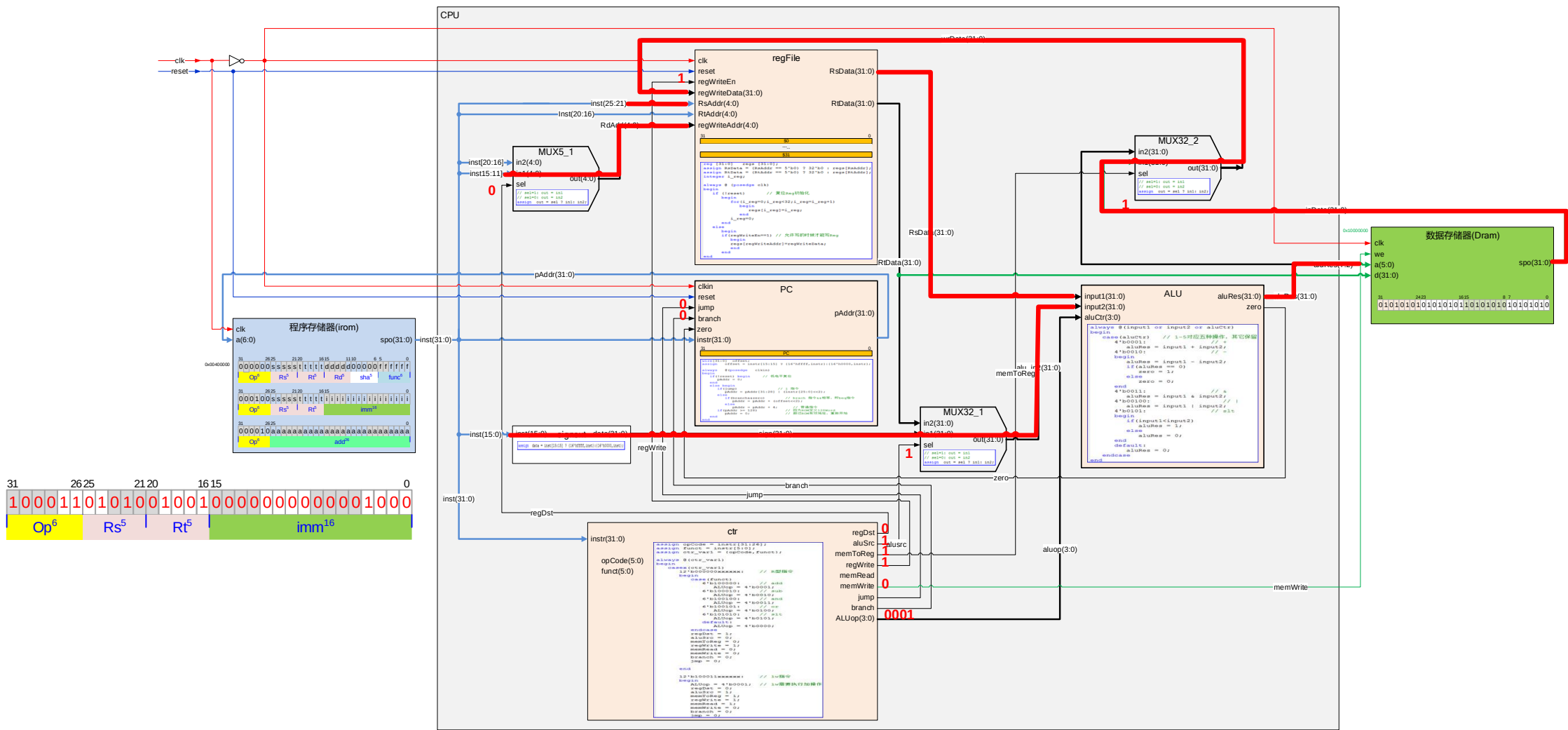
Op-Code	Imm
000010	iiiiiiiiiiiiiiiiiiiiiiiiiiii

Load the PC with an address formed by concatenating the first 4-bits of the current PC with the value in the 26-bit immediate field shifted left 2-bits.

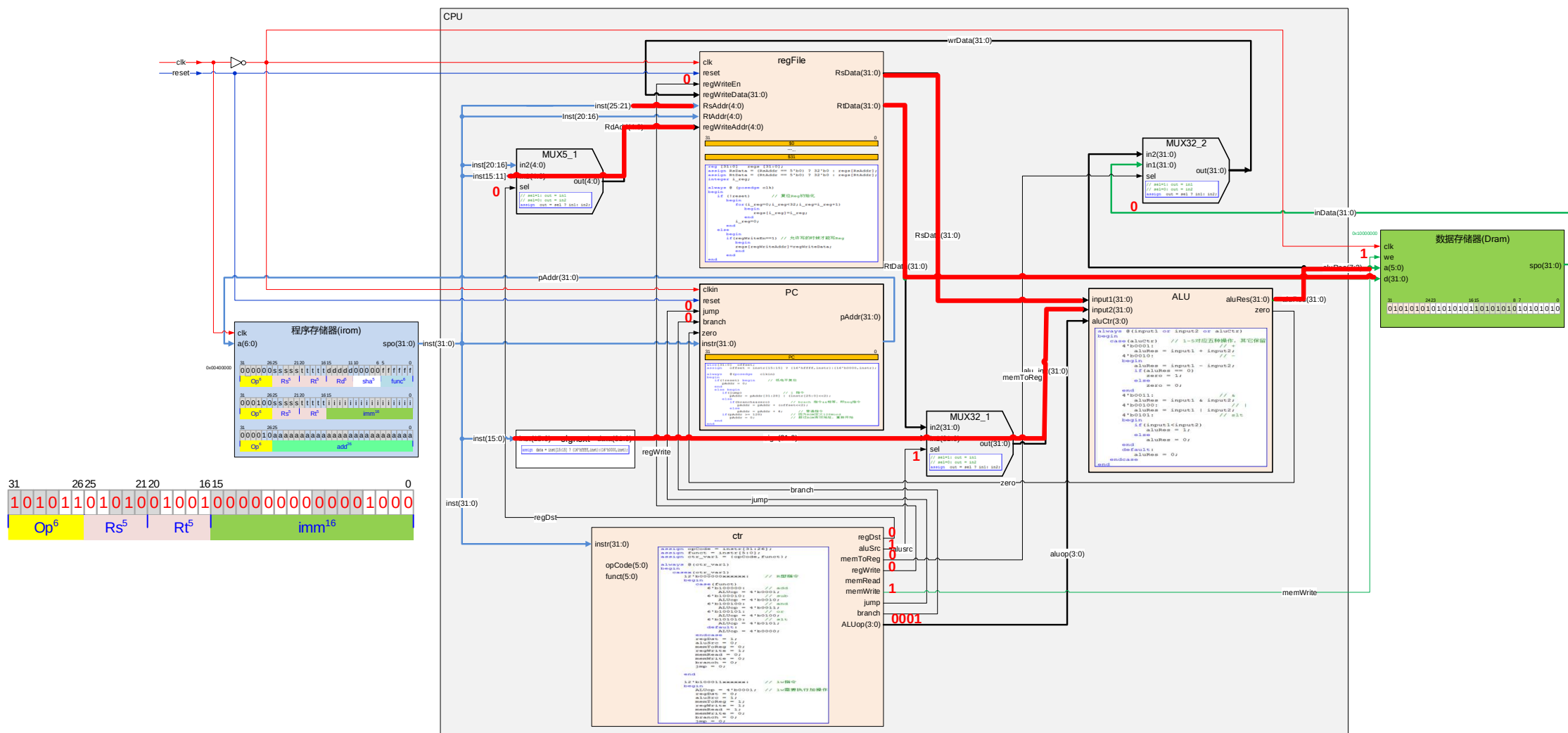
► 简单微处理器的基本构成（数据流：add \$t1, \$t2, \$t3 指令）



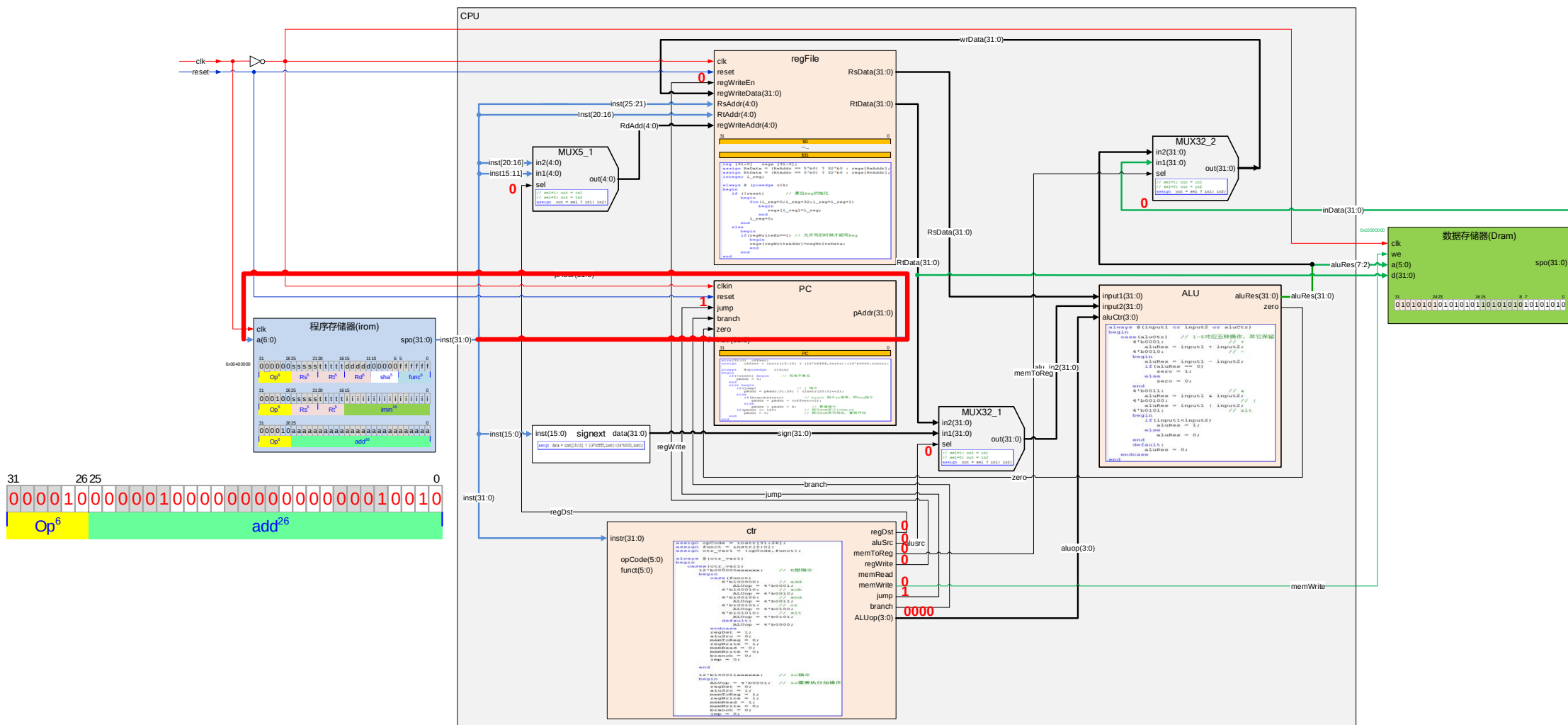
简单微处理器的基本构成（数据流：lw \$t1,8(\$t2)指令）



► 简单微处理器的基本构成（数据流：sw \$t1,8(\$t2)指令）



► 简单微处理器的基本构成（数据流：j xxxx指令）



▶ 实验内容

- 目的
- 任务及时间安排
- 报告要求

▶ 原理回顾

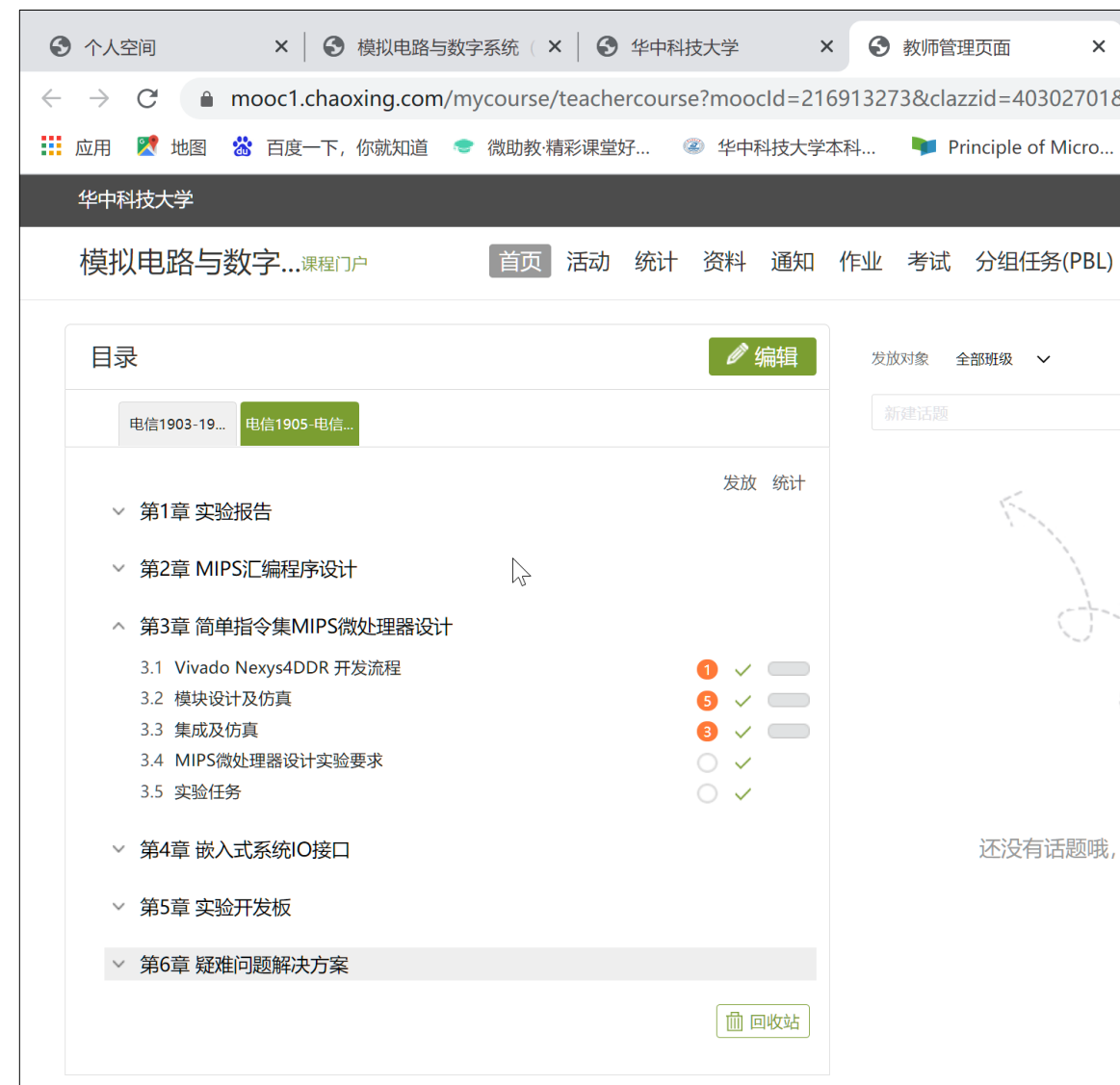
- 计算机工作原理
- 简单微处理器的基本构成

▶ Vivado软件使用

- 模块设计
- 功能仿真

▶ 详见超星资源视频(9个)

- Vivado开发流程 : 1
- 模块设计及仿真 : 5
 - 寄存器模块设计及仿真
 - ALU模块设计及仿真
 - 指令、数据存储器
 - Mars导出汇编指令机器码
 - 主控制器实现
- 集成及仿真 : 3
 - 顶层模块实现
 - 处理器仿真一
 - 处理器仿真二



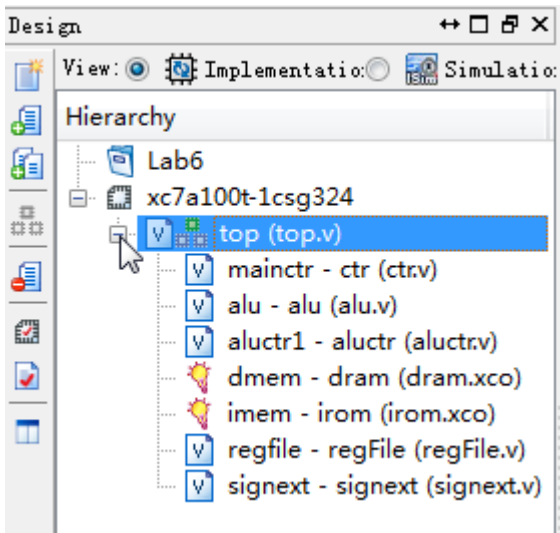
► 顶层模块

- 采用Verilog实现：top.v
 - 定义信号线
 - 模块实例化，连接模块
 - 时钟信号线

```
21 module top(  
22     input clkin,  
23     input reset  
24 );  
25  
26 //wire for ROM  
27 wire[31:0] inst;  
28  
29 //wire for controller  
30 wire      regdst,jump,branch,memread,memwrite,memtoreg;  
31 wire[3:0] aluop;  
32 wire      alusrc,regwrite;  
33  
34 //wire for aluunit  
35 wire      zero;  
36 wire[31:0] aluRes;  
37 wire[31:0] alu_in2;           // ALU数据输入之前的多路器输出  
38  
39 //wire for memory  
40 wire[31:0] inData;  
41  
42 //wire for register  
43 wire[31:0] RsData,RtData;  
44 wire[4:0]  RdAddr;  
45 wire[31:0] wrData;  
46  
47 //wire for PC  
48 wire[31:0] pAddr;  
49  
50 //wire for ext  
51 wire[31:0] sign;  
52
```

► 顶层模块

- 采用Verilog实现：top.v
 - 定义信号线
 - 模块实例化，连接模块



- 时钟信号线

```
52
53 irom imem(
54     .a(pAddr[8:2]),
55     .clk(clkin),          // 上升沿读指令
56     .spo(inst));
57
58 PC PCnt(
59     .clkin(!clkin),       // 下降沿改变PC
60     .reset(reset),
61     .instr(inst),
62     .jump(jump),
63     .branch(branch),
64     .zero(zero),
65     .pAddr(pAddr));
66
67 ctr mainctr(
68     .instr(inst),
69     .ALUop(aluop),
70     .regDst(regdst),
71     .aluSrc(alusrc),
72     .memToReg(memtoreg),
73     .regWrite(regwrite),
74     .memRead(memread),
75     .memWrite(memwrite),
76     .branch(branch),
77     .jmp(jump));
78
79 alu alu(
80     .input1(RsData),
81     .input2(alu_in2),
82     .aluCtr(aluop),
83     .aluRes(aluRes),
84     .zero(zero));
85
86 // ALU input2 数据多路器
87 mux32 max32_1(
88     .in1(sign),
89     .in2(RtData),
90     .out(alu_in2),
91     .sel(alusrc));
```

```
92
93 dram dmem(
94     .a(aluRes[7:2]),
95     .d(RtData),
96     .clk(!clkin),        // 下降沿写数据
97     .we(memwrite),
98     .spo(inData));
99
100 // Reg 写数据多路器
101 mux32 max32_2(
102     .in1(inData),
103     .in2(aluRes),
104     .out(wrData),
105     .sel(memtoreg));
106
107 regFile regfile(
108     .RsAddr(inst[25:21]),
109     .RtAddr(inst[20:16]),
110     .clk(!clkin),        // 下降沿写Reg
111     .reset(reset),
112     .regWriteAddr(RdAddr),
113     .regWriteData(wrData),
114     .regWriteEn(regwrite),
115     .RsData(RsData),
116     .RtData(RtData));
117
118 // Reg 地址选择多路器
119 mux5 max5_1(
120     .in1(inst[15:11]),
121     .in2(inst[20:16]),
122     .out(RdAddr),
123     .sel(regdst));
124
125 // 符号数扩展
126 signext signext(
127     .inst(inst[15:0]),
128     .data(sign));
129
130 endmodule
```

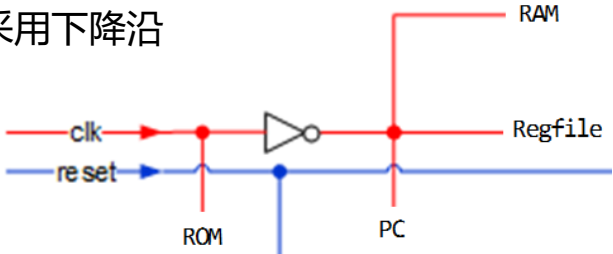
► 顶层模块

• 采用Verilog实现：top.v

- 定义信号线
- 模块实例化，连接模块
- 时钟信号线
 - 单周期

- 执行指令时不能取指令，取指令由ROM输出决定，执行指令包括改变PC的值、写Regs、写RAM等
- 因此改变PC的值、写Regs、写RAM等与从ROM读取指令不能同时进行，这里采用时钟的两个不同边沿实现

- 一个采用上升沿
- 一个采用下降沿



```
52
53 irom imem(
54     .a(pAddr[8:2]),
55     .clk(clkin),          // 上升沿读指令
56     .spo(inst));
57
58 PC PCnt(
59     .clkin(!clkin),       // 下降沿改变PC
60     .reset(reset),
61     .instr(inst),
62     .jump(jump),
63     .branch(branch),
64     .zero(zero),
65     .pAddr(pAddr));
66
67 ctr mainctr(
68     .instr(inst),
69     .ALUop(aluop),
70     .regDst(regdst),
71     .aluSrc(alusrc),
72     .memToReg(memtoreg),
73     .regWrite(regwrite),
74     .memRead(memread),
75     .memWrite(memwrite),
76     .branch(branch),
77     .jump(jump));
78
79 alu alu(
80     .input1(RsData),
81     .input2(alu_in2),
82     .aluCtr(aluop),
83     .aluRes(aluRes),
84     .zero(zero));
85
86 // ALU input2 数据多路器
87 mux32 max32_1(
88     .in1(sign),
89     .in2(RtData),
90     .out(alu_in2),
91     .sel(alusrc));
```

```
92
93 dram dmem(
94     .a(aluRes[7:2]),
95     .d(RtData),
96     .clk(!clkin),       // 下降沿写数据
97     .we(memwrite),
98     .spo(inData));
99
100 // Reg 写数据多路器
101 mux32 max32_2(
102     .in1(inData),
103     .in2(aluRes),
104     .out(wrData),
105     .sel(memtoreg));
106
107 regFile regfile(
108     .RsAddr(inst[25:21]),
109     .RtAddr(inst[20:16]),
110     .clk(!clkin),       // 下降沿写Reg
111     .reset(reset),
112     .regWriteAddr(RdAddr),
113     .regWriteData(wrData),
114     .regWriteEn(regwrite),
115     .RsData(RsData),
116     .RtData(RtData));
117
118 // Reg 地址选择多路器
119 mux5 max5_1(
120     .in1(inst[15:11]),
121     .in2(inst[20:16]),
122     .out(RdAddr),
123     .sel(regdst));
124
125 // 符号数扩展
126 signext signext(
127     .inst(inst[15:0]),
128     .data(sign));
129
130 endmodule
```



► 实验内容

- 目的
- 任务及时间安排
- 报告要求

► 原理回顾

- 计算机工作原理
- 简单微处理器的基本构成

► Vivado软件使用

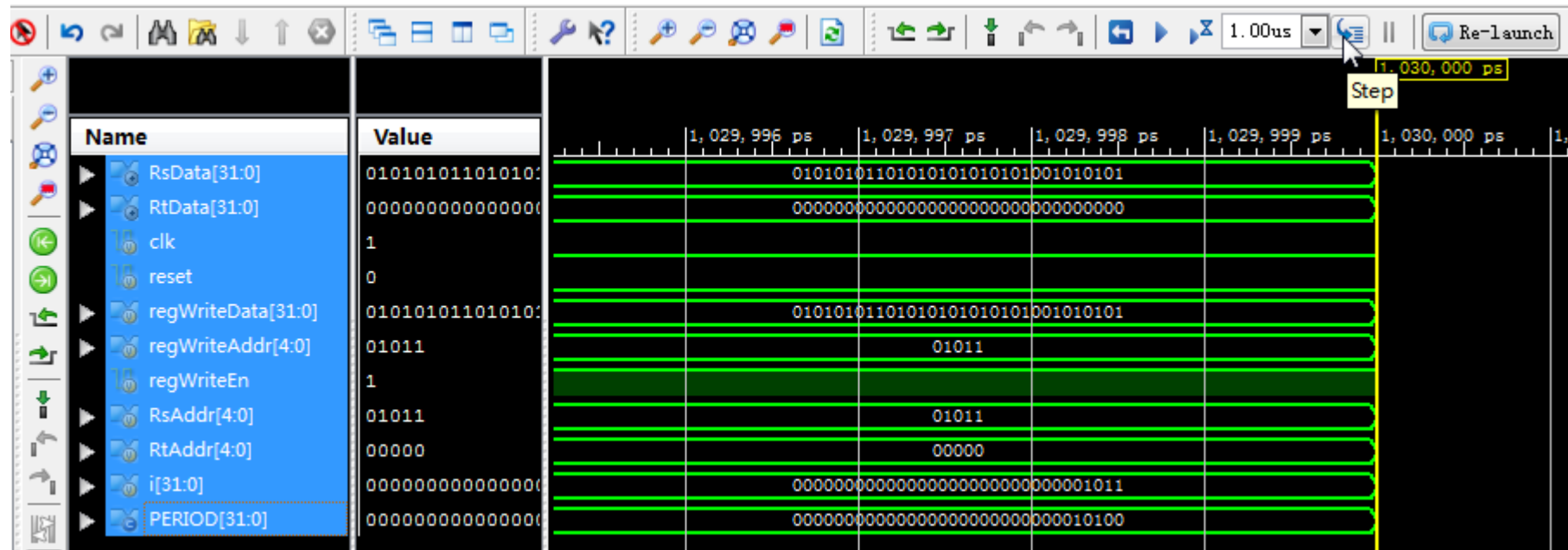
- 模块设计
- 功能仿真

► 仿真

- 子模块仿真
 - 寄存器组仿真 (学会仿真步骤)
 - ...
- 顶层仿真 (掌握仿真结果的查看)

► 寄存器组仿真

- 观察结果
 - Step



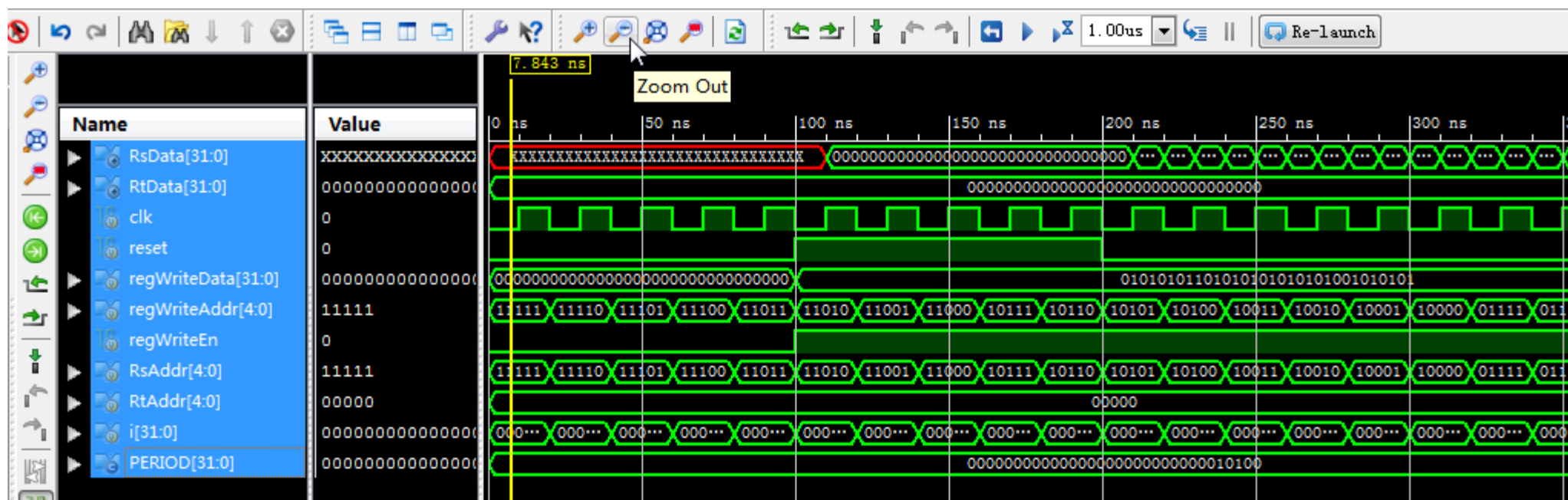
- Run for the time specified on the toolbar



► 寄存器组仿真

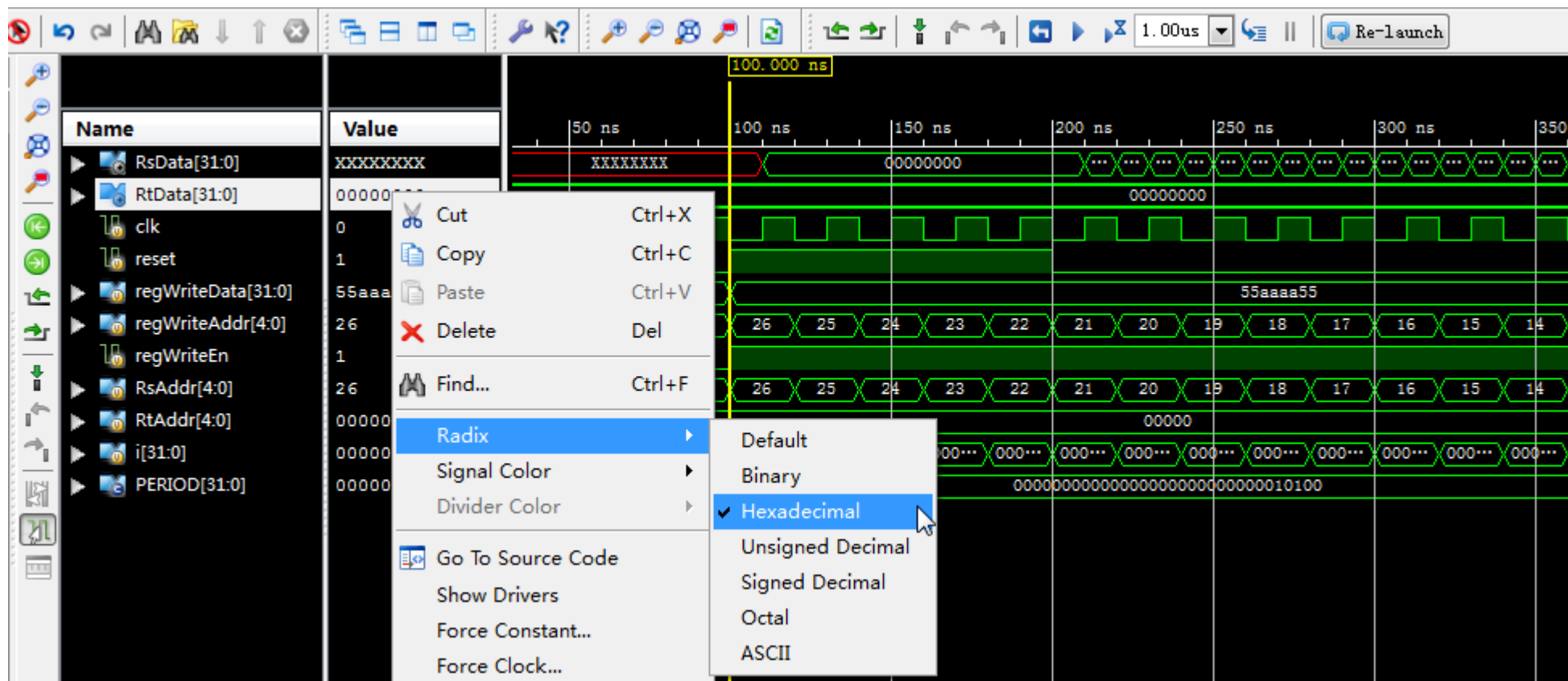
• 观察结果

- Run for the time specified on the toolbar
- Zoom Out / Zoom In



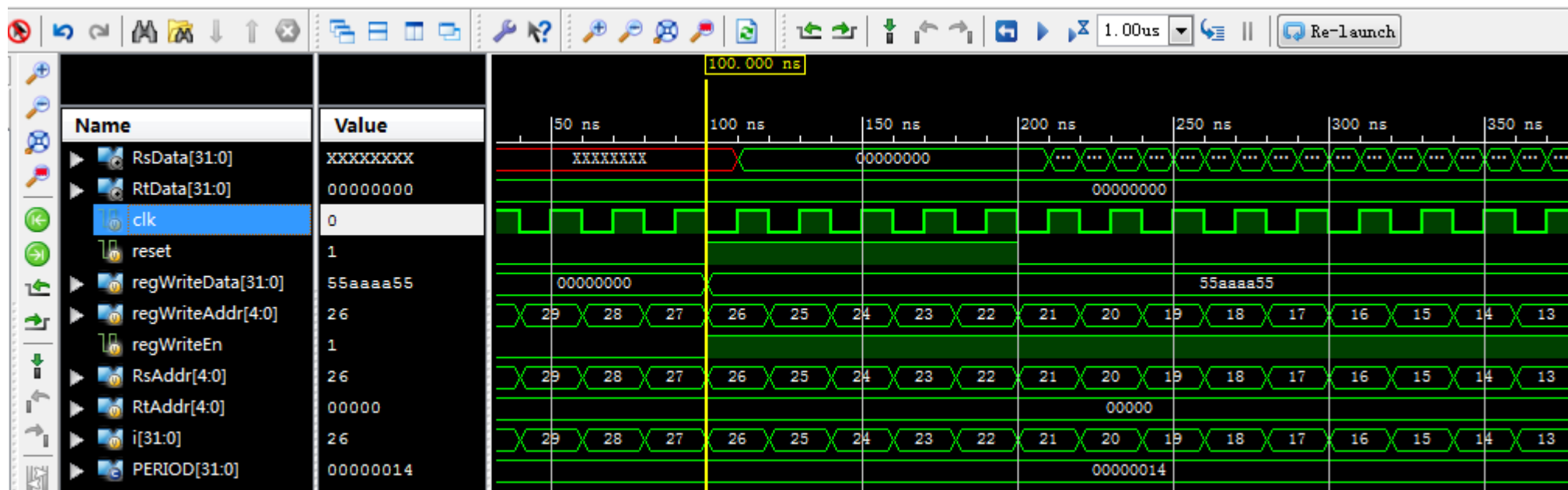
► 寄存器组仿真

- 观察结果
 - 显示格式调整



► 寄存器组仿真

- 观察结果
 - 结果分析



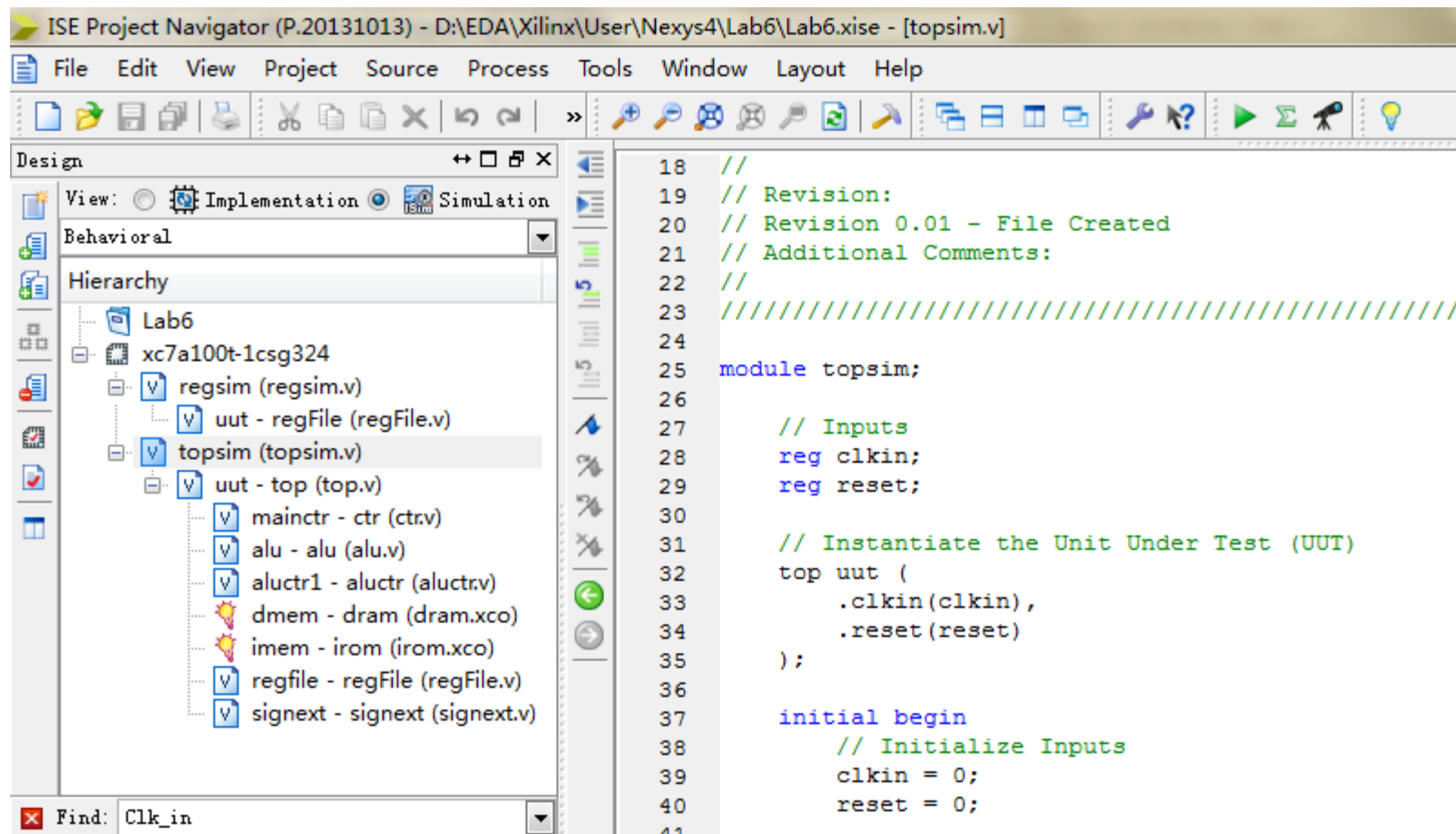
► 其它模块功能仿真

- 控制器模块
- ALU模块
- ROM模块
- RAM模块
- ...

自行完成这些模块的功能仿真，保证模块功能是正确的；之后可以进行Top的功能仿真；或者如发现Top功能仿真不对，可以返回来仿真各个模块功能，找出错误所在。

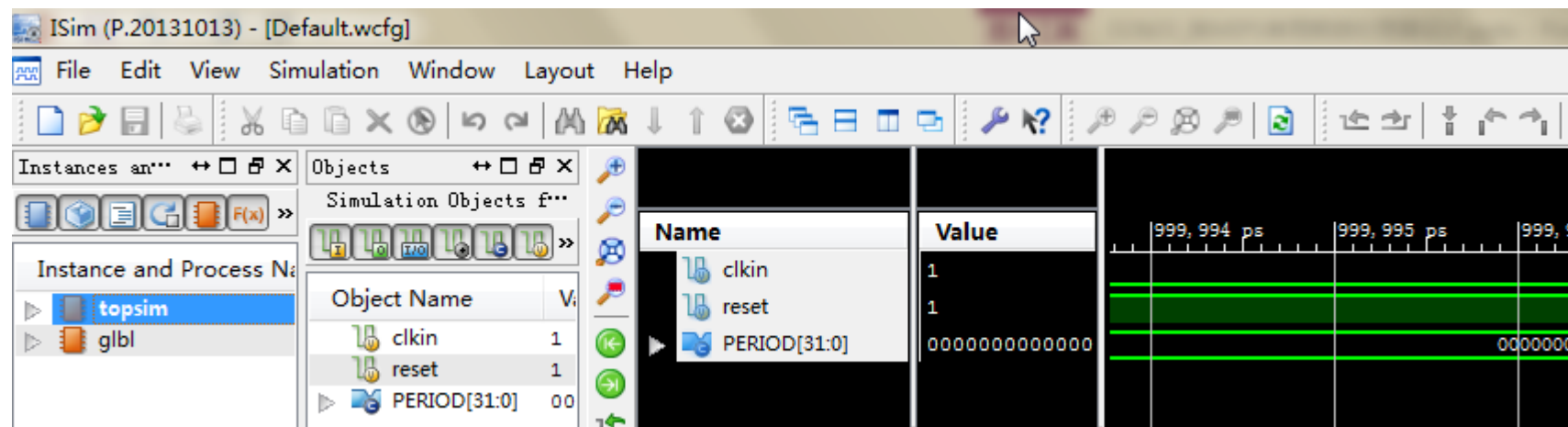
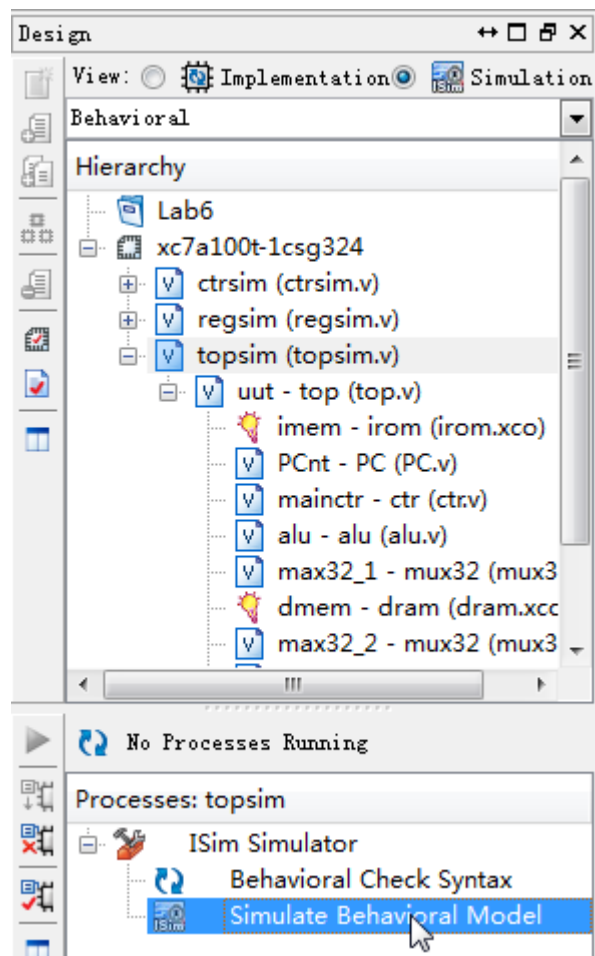
► 顶层仿真

- 自动生成激励代码topsim.v



► 顶层仿真

- 运行仿真&查看结果



► 顶层仿真

- 观察结果：程序ROM、数据RAM是否正确？

Address: Columns: Address Radix: Hexadecimal Value Radix: Hexadec

	0	1	2	3	4	5	6	7
0x0	00432020	8C440004	AC420008	00831022	00831025	00831024	0083102A	10830002
0x8	08000000	8C620000	08000000	00000000	00000000	00000000	00000000	00000000
0x10	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x18	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x20	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x28	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x30	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x38	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x40	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x48	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x50	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x58	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x60	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x68	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x70	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0x78	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

前面步骤中通过IP Core建立的ROM中，用户程序11条指令的机器码已经存入在ROM里。

[编辑1] - UltraEdit

文件(F) 编辑(E) 搜索(S) 插入(N) 工程(P) 视图(V) 格式(T) 列(L) 宏(M) 脚本(I) 高级(A) 窗口(W) 帮助(H)

列模式(C) Alt+C

插入/填充列(I)...

删除列(D)...

剪切列(U)...

插入数字(N)...

列/选区求和(S)...

转为固定列宽(X)...

转为字符分隔(D)...

左对齐(L)

居中(C)

右对齐(R)

	0	1	2	3	4	5	6	7
1	[00400024]	00432020	add \$4, \$2, \$3					
2	[00400028]	8c440004	lw \$4, 4(\$2)					
3	[0040002c]	ac420008	sw \$2, 8(\$2)					
4	[00400030]	00831022	sub \$2, \$4, \$3					
5	[00400034]	00831025	or \$2, \$4, \$3					
6	[00400038]	00831024	and \$2, \$4, \$3					
7	[0040003c]	0083102a	slt \$2, \$4, \$3					
8	[00400040]	10830002	beq \$4, \$3, exit					
9	[00400044]	08100009	j 0x00400024 [main]					
10	[00400048]	8c620000	lw \$2, 0(\$3)					
11	[0040004c]	08100009	j 0x00400024 [main]					

ISim (P.20131013) - [Default.wcfg*]

File Edit View Simulation Window Layout Help

Memory

/topsim/uut/dmem/inst/ram_data[63:0,31:0]

/topsim/uut/dmem/inst/ram_data_tmp[63:0,31:0]

/topsim/uut/imem/inst/ram_data[127:0,31:0]

/topsim/uut/imem/inst/ram_data_tmp[127:0,31:0]

/topsim/uut/imem/inst/ram_data[127:0,31:0]

Instances and Processes Memory Source Files

▶ 顶层仿真

- 观察结果：程序ROM、数据RAM是否正确？

Address: Columns: auto Address Radix: Hexadecimal Value Radix: Hexadecimal

	0	1	2	3	4	5	6	7
0x0	55555555	55555555	00000002	55555555	55555555	55555555	55555555	55555555
0x8	55555555	55555555	55555555	55555555	55555555	55555555	55555555	55555555
0x10	55555555	55555555	55555555	55555555	55555555	55555555	55555555	55555555
0x18	55555555	55555555	55555555	55555555	55555555	55555555	55555555	55555555
0x20	55555555	55555555	55555555	55555555	55555555	55555555	55555555	55555555
0x28	55555555	55555555	55555555	55555555	55555555	55555555	55555555	55555555
0x30	55555555	55555555	55555555	55555555	55555555	55555555	55555555	55555555
0x38	55555555	55555555	55555555	55555555	55555555	55555555	55555555	55555555

前面步骤中通过IP Core建立的RAM中，大都为0x55555555（个别因为代码的执行发生了改变）。

Memory

- /topsim/uut/dmem/inst/ram_data[63:0,31:0]
- /topsim/uut/dmem/inst/ram_data tmp[63:0,31:0]
- /topsim/uut/imem/inst/ram_data[63:0,31:0]
- /topsim/uut/imem/inst/ram_data tmp[127:0,31:0]
- /topsim/uut/regfile/regs[31:0,31:0]

Distributed Memory Generator

View Documents

IP Symbol

LogiCORE

Distributed Memory Generator

Load COE File

If desired the initial memory content can be set by using a COE file. This Initialisation File (MIF).

Coefficients File : no_coe_file_loaded Browse... Show...

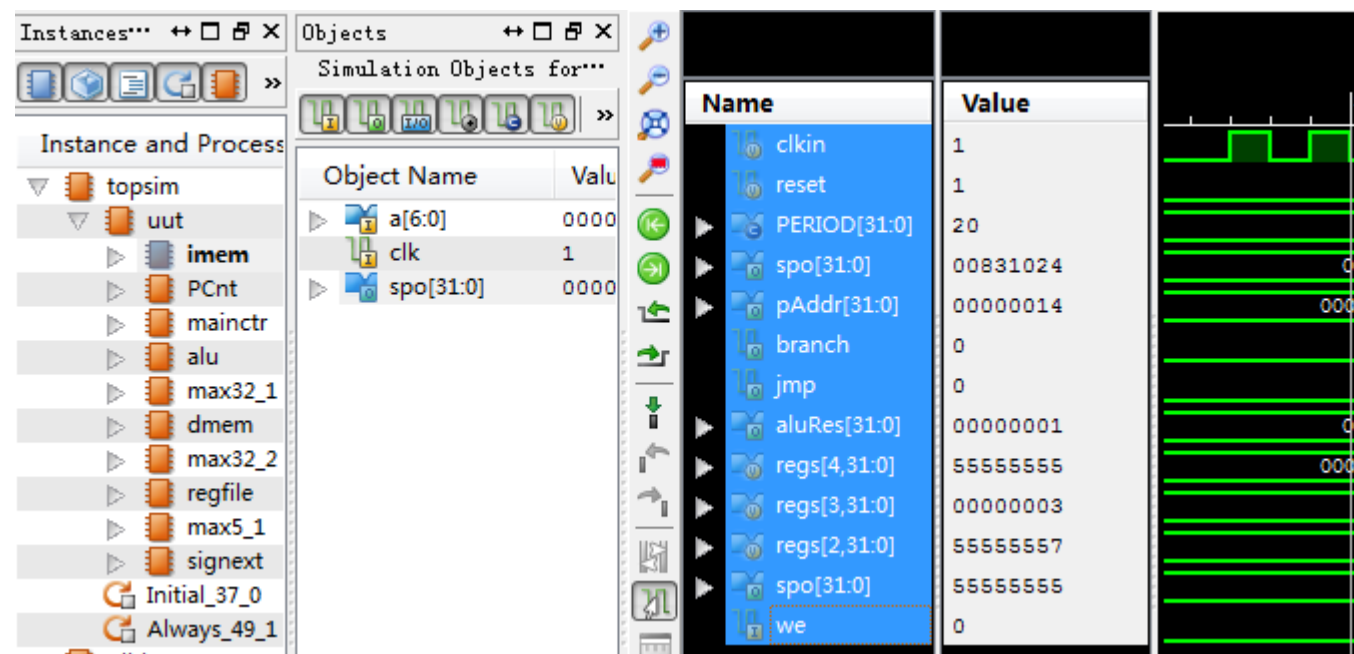
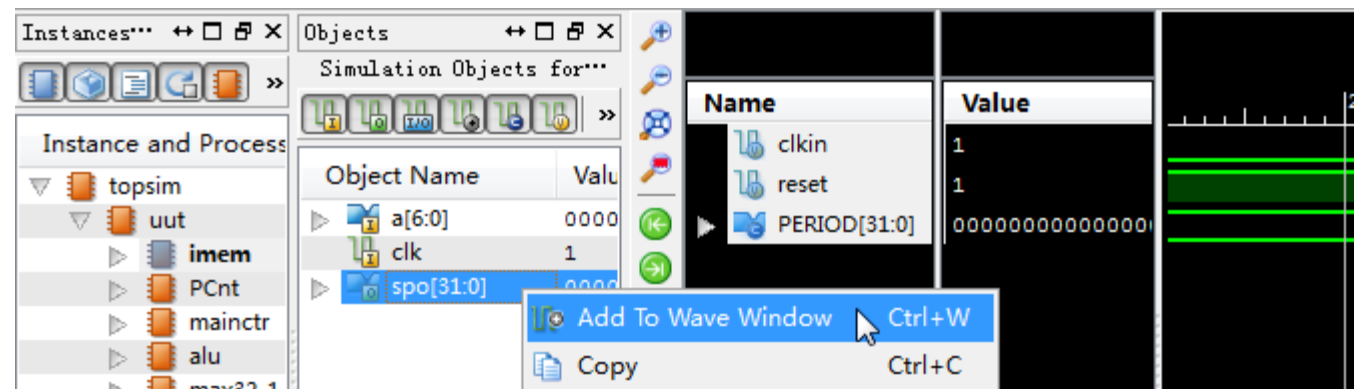
COE Options

Default Data : 55555555 Radix : 16

Instances and Processes Memory Source Files

► 顶层仿真

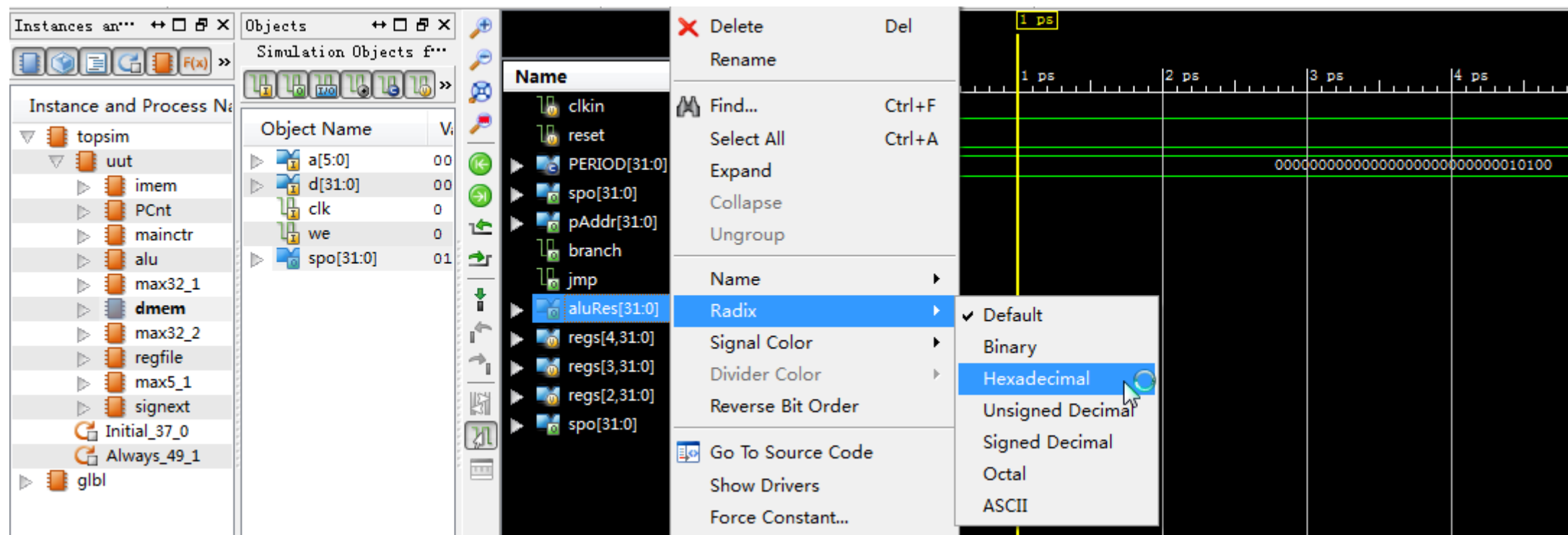
- 观察结果：加入需要观测的信号
 - 程序信息总线：imem/spo[31:0]；
 - 程序地址总线：PCnt/pAddr[31:0]；
 - 控制信息：mainctr/branch、jmp；
 - Alu结果：alu/aluRes[31:0]；
 - \$4/\$3/\$2：regfile/regs[4,31:0]、regfile/regs[3,31:0]、regfile/regs[2,31:0]；
 - RAM数据总线：dmem/spo[31:0]
 - RAM写控制线：dmem/we



► 顶层仿真

• 观察结果：

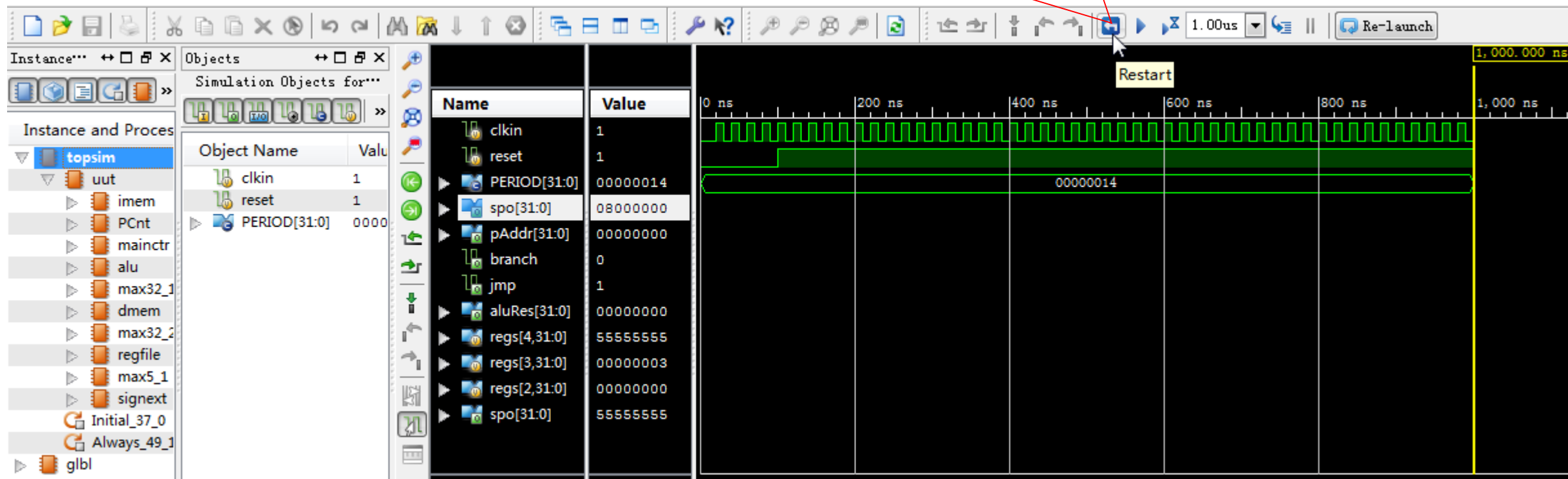
- 调整观测信号的**显示格式**（总线均按照16进制显示）
- Zoom in / Zoom out，使得整个窗口能够显示多个时钟周期信号



► 顶层仿真

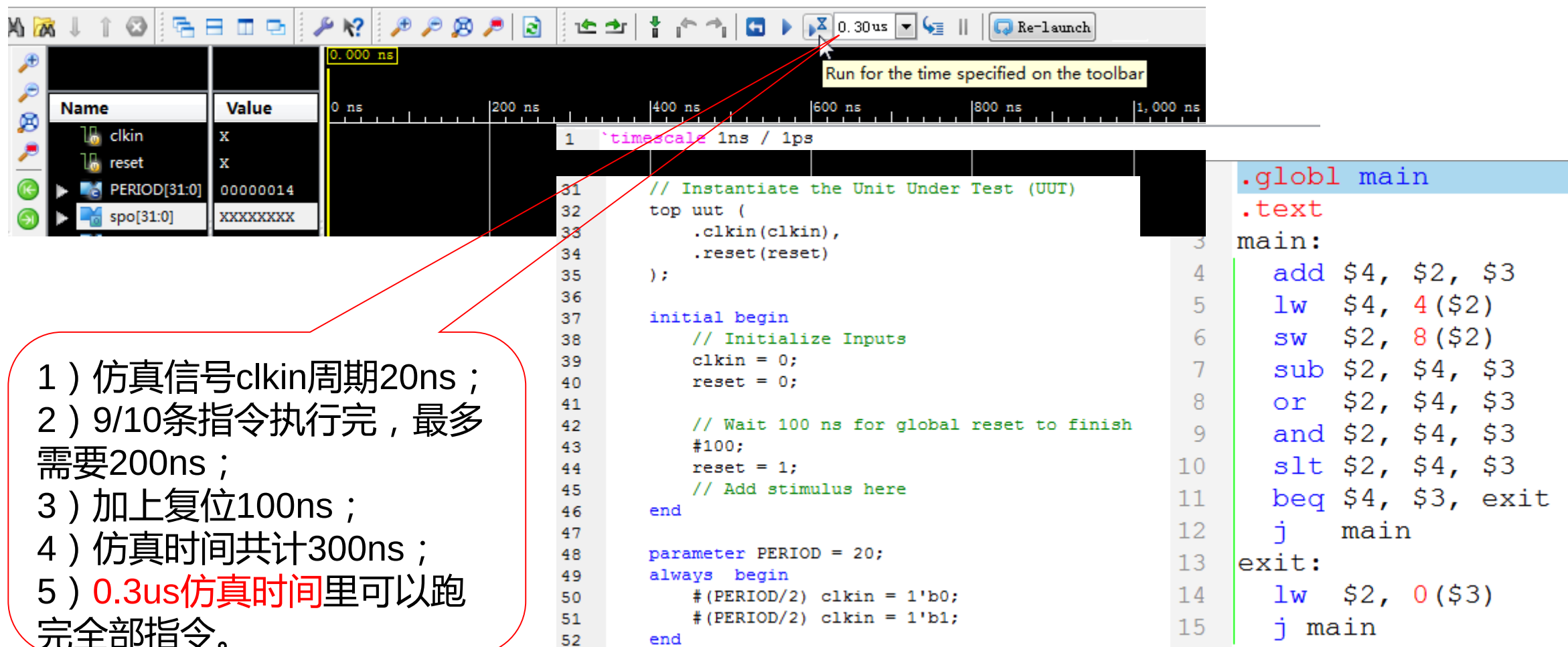
- 观察结果：Restart

1) 一进入仿真界面，就自动执行了1.00us时间，仅仅显示了top模块的信号变化；后加入进来的信号变化过程没有显示。所以先**Restart**，重新开始仿真。



► 顶层仿真

- 观察结果：修改仿真时间，然后 Run for the time specified on the toolbar



The screenshot displays the Vivado IDE interface. On the left, a table lists signals and their values:

Name	Value
clk	X
reset	X
PERIOD[31:0]	00000014
spo[31:0]	XXXXXXXX

The top toolbar shows a time scale of 0.30us and a 'Run for the time specified on the toolbar' button. A red arrow points from this button to a list of simulation steps.

1) 仿真信号clk周期20ns ;
2) 9/10条指令执行完，最多需要200ns ;
3) 加上复位100ns ;
4) 仿真时间共计300ns ;
5) 0.3us仿真时间里可以跑完全部指令。

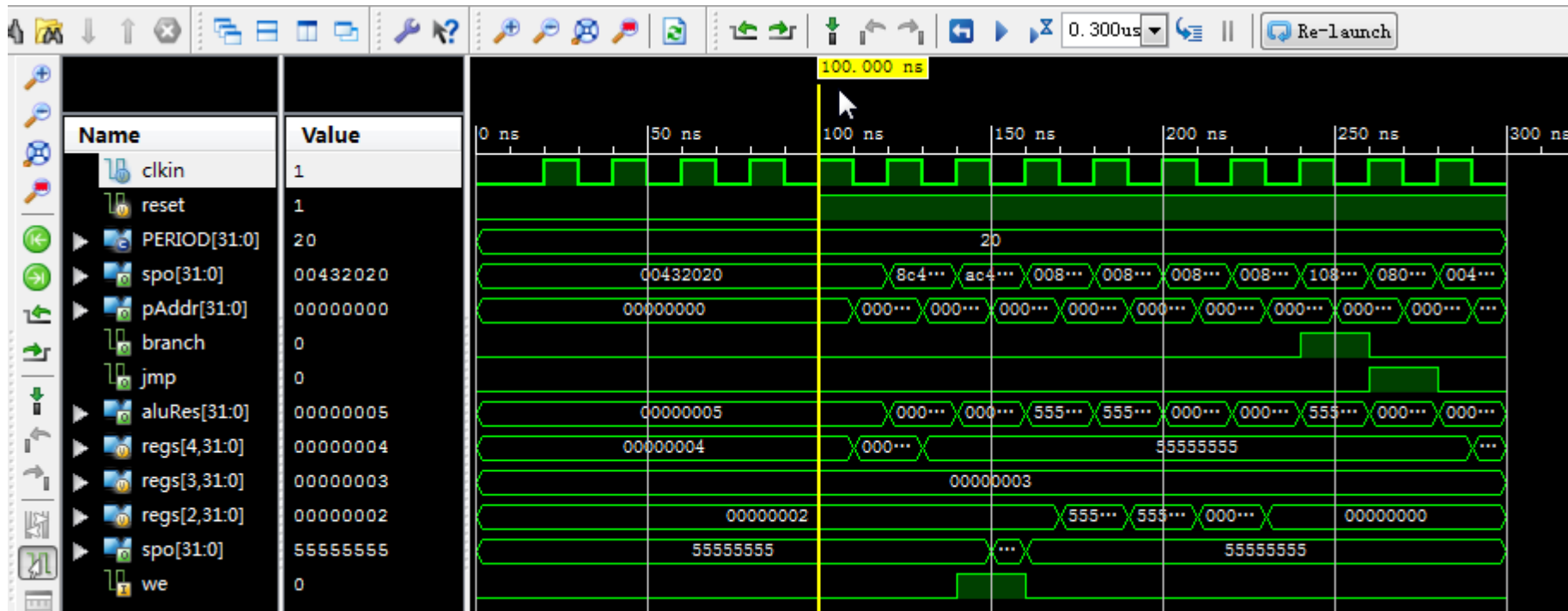
The main editor shows Verilog code for a Unit Under Test (UUT) and assembly code for a global main function.

```
1 `timescale 1ns / 1ps
31 // Instantiate the Unit Under Test (UUT)
32 top uut (
33     .clk(clk),
34     .reset(reset)
35 );
36
37 initial begin
38     // Initialize Inputs
39     clk = 0;
40     reset = 0;
41
42     // Wait 100 ns for global reset to finish
43     #100;
44     reset = 1;
45     // Add stimulus here
46 end
47
48 parameter PERIOD = 20;
49 always begin
50     #(PERIOD/2) clk = 1'b0;
51     #(PERIOD/2) clk = 1'b1;
52 end
```

```
.globl main
.text
main:
4     add $4, $2, $3
5     lw $4, 4($2)
6     sw $2, 8($2)
7     sub $2, $4, $3
8     or $2, $4, $3
9     and $2, $4, $3
10    slt $2, $4, $3
11    beq $4, $3, exit
12    j main
13 exit:
14    lw $2, 0($3)
15    j main
```

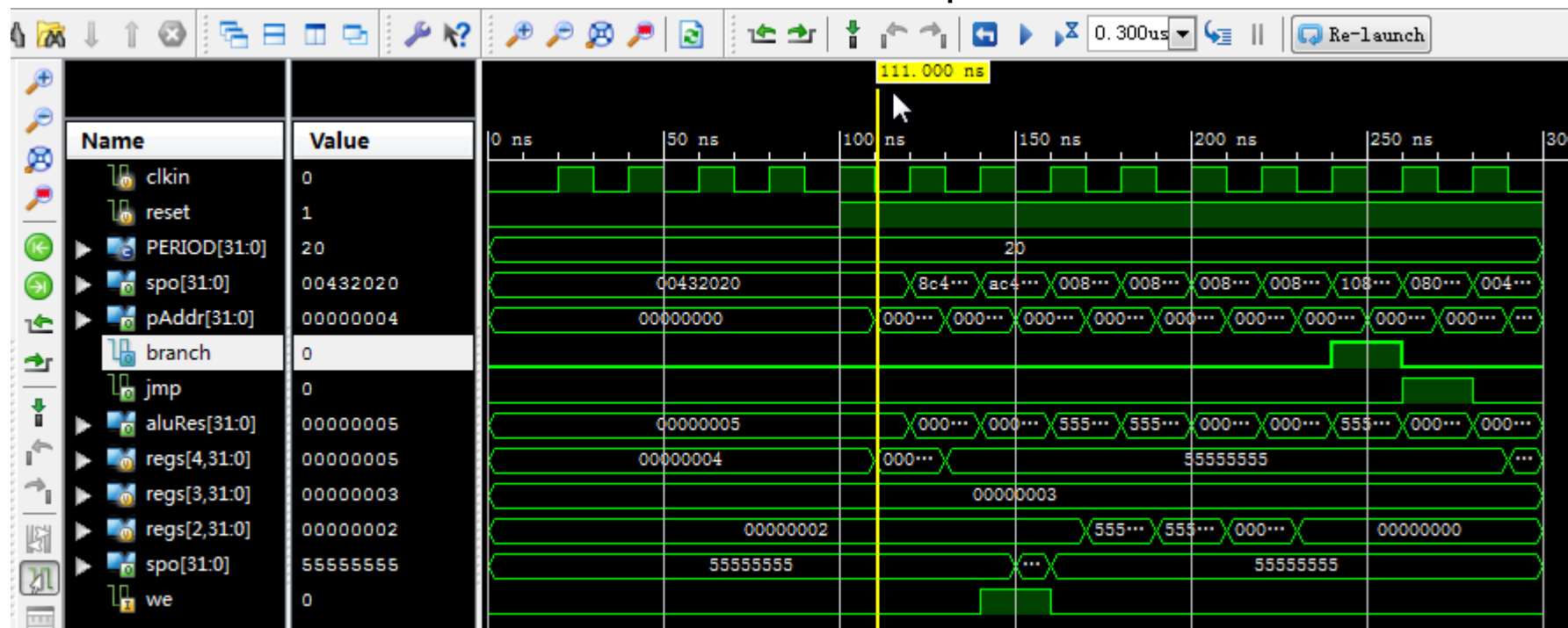
► 顶层仿真

- 观察结果：复位后**第1个**时钟周期，应该执行 add \$4, \$2, \$3 （ 机器码:00432020 ）
 - 周期时钟**上升沿（100ns处）**从ROM读指令，PC指针pAddr = 0x0000 0000，控制器根据机器码产生对应的控制信号。
 - 此时\$4, \$3, \$2寄存器的值为复位初始化值：4,3,2。



► 顶层仿真

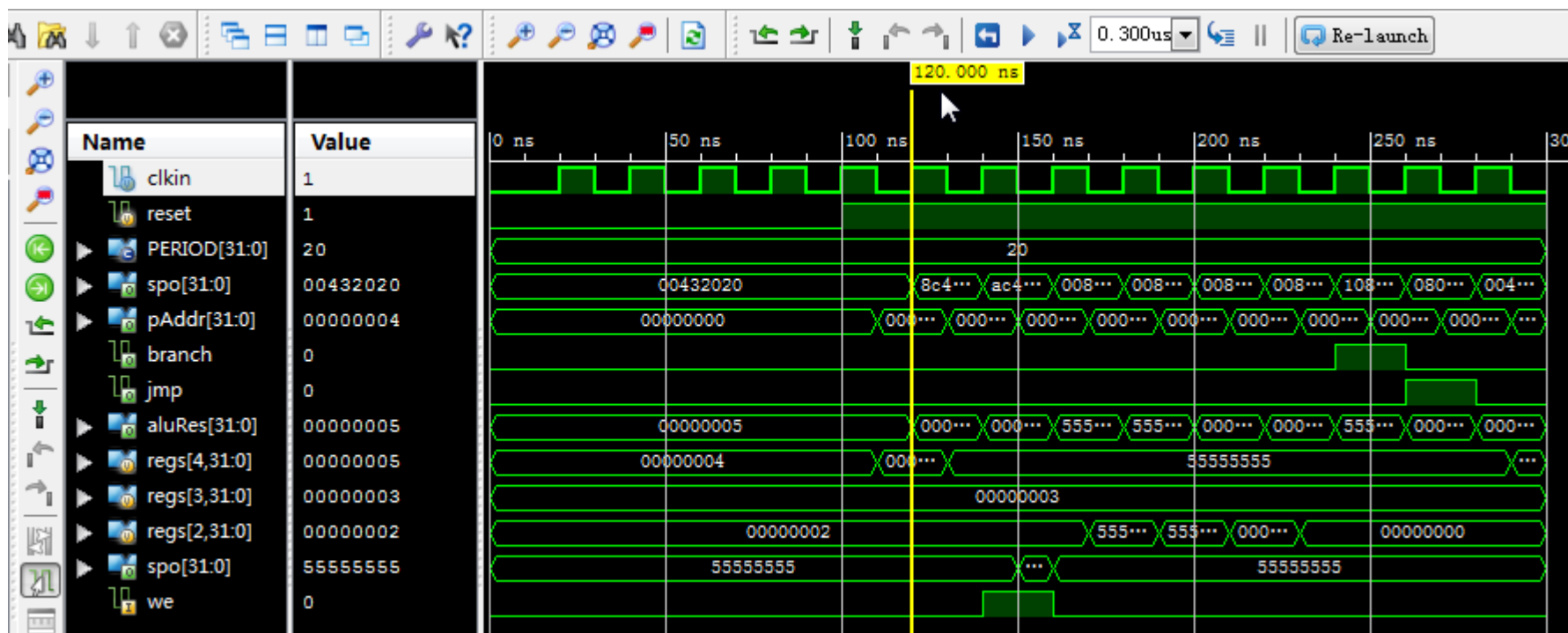
- 观察结果：复位后**第1个**时钟周期，应该执行 add \$4, \$2, \$3 （ 机器码:00432020 ）
 - 周期时钟**下降沿**（ **110ns处** ）执行指令，写Reg、改变PC指针，执行后，如**111ns**处：
 - 机器码spo = 00432020
 - \$2 + \$3 → \$4 , 2+3=5 , \$4 = 5 （ 之前 \$4, \$3, \$2的初始化值：4,3,2 ）, 从仿真可知：Alu的计算结果aluRes为5 , \$4的值也为5；PC指针pAddr = 0x0000 0004 , 指向了下一个地址。



由此可见
， add 指
令被正确
执行。

► 顶层仿真

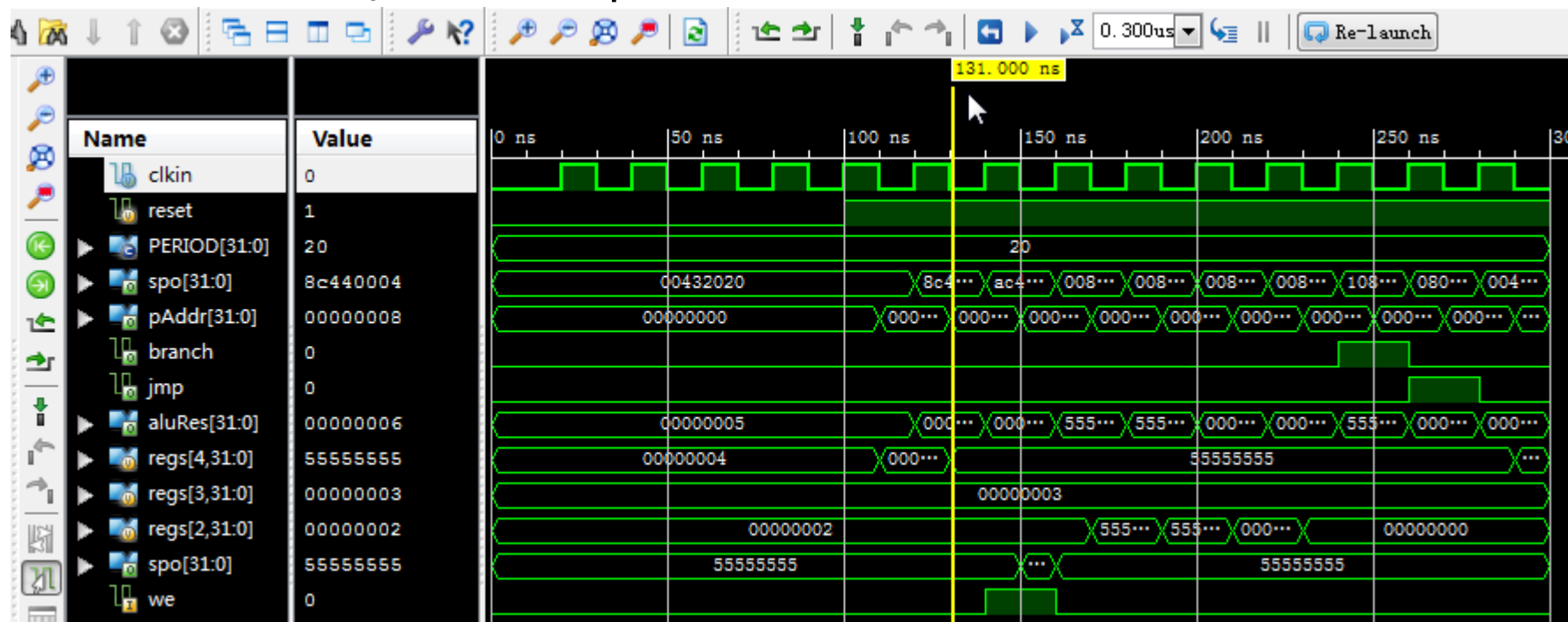
- 观察结果：复位后**第2个**时钟周期，应该执行 `lw $4, 4($2)`（机器码:8c440004）
 - 周期时钟**上升沿（120ns处）**从ROM读指令，PC指针pAddr = 0x0000 0004，控制器根据机器码产生对应的控制信号。
 - 此时\$4, \$2寄存器的值为：5, 2。



► 顶层仿真

- 观察结果：复位后**第2个**时钟周期，应该执行 lw \$4, 4(\$2)（机器码:8c440004）
 - 周期时钟**下降沿**（**130ns处**）执行指令，写Reg、改变PC指针，执行后，如**131n**处：
 - 机器码spo = 8c440004
 - (\$2 + 4) → \$4，（之前 \$4, \$2的值：5,2），从仿真可知：\$4的值变为55555555（即地址为6的字单元中的值）；PC指针pAddr = 0x0000 0008，指向了下一个地址。

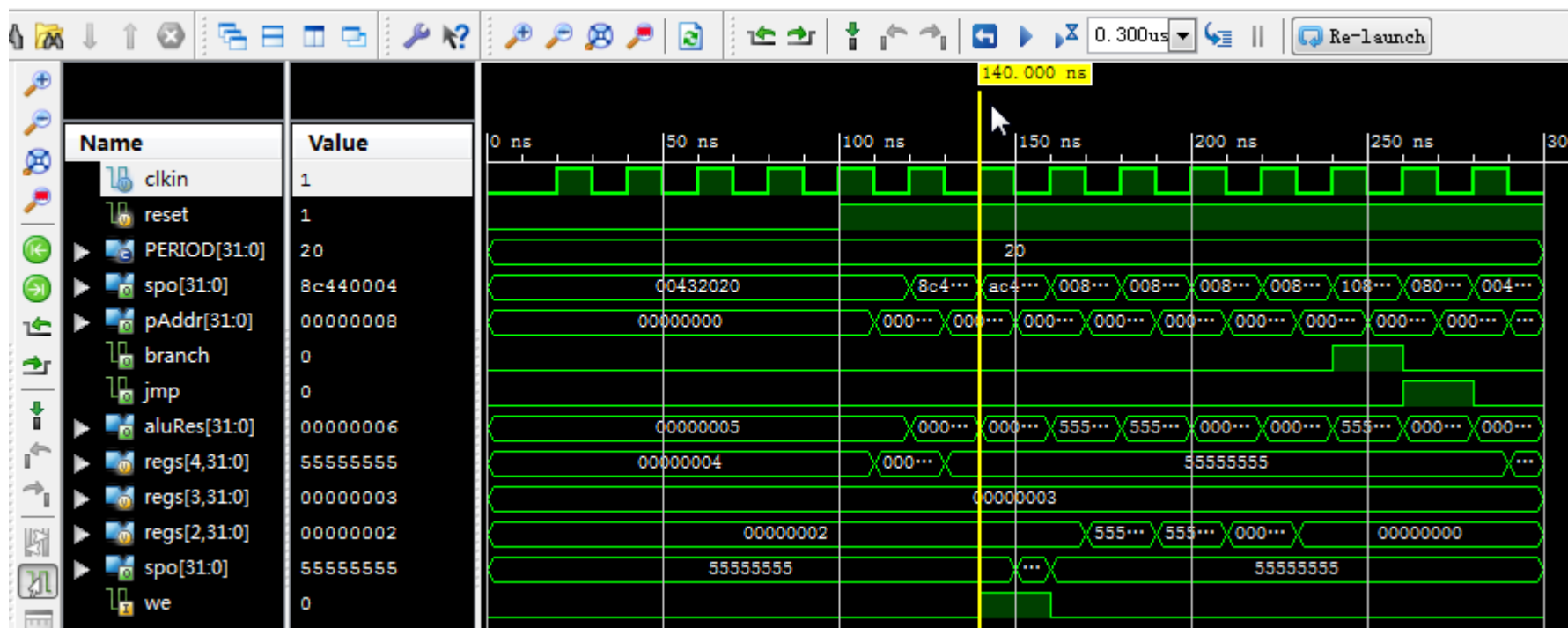
实际为地址4的字单元，因为ram硬件按照字读写：
.a(aluRes[7:2]),



由此可见，lw 指令被正确执行。

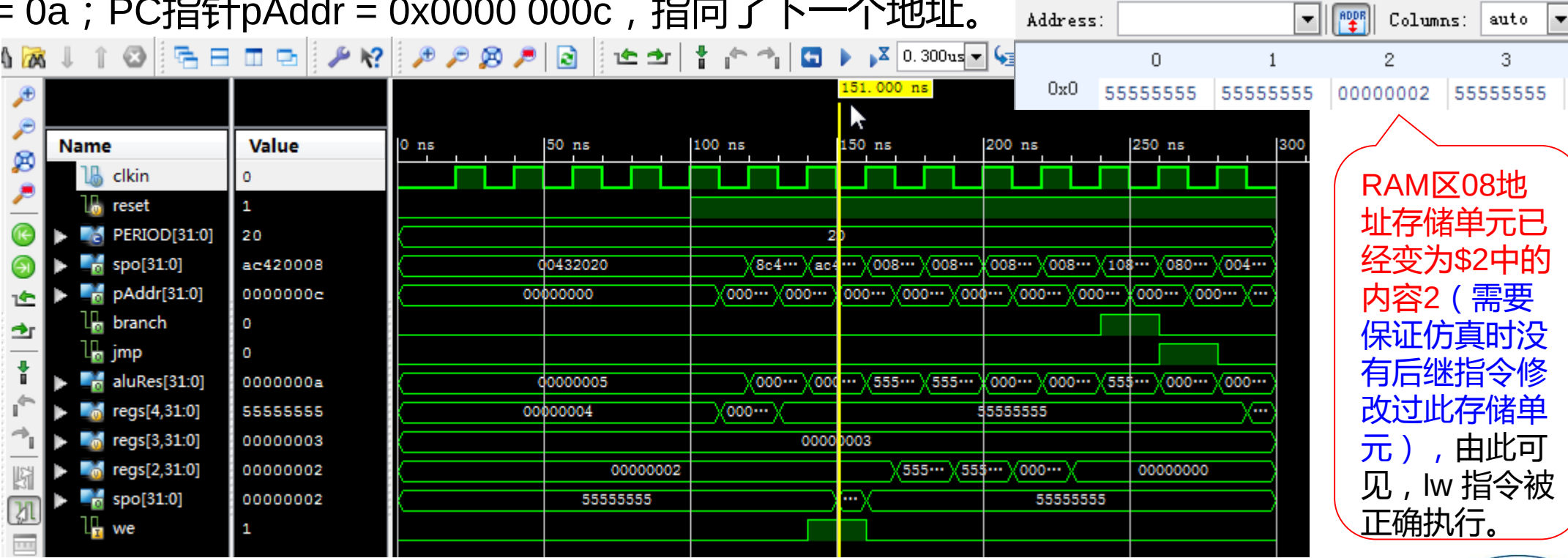
► 顶层仿真

- 观察结果：复位后**第3个**时钟周期，应该执行 `sw $2, 8($2)`（机器码:ac420008）
 - 周期时钟**上升沿（140ns处）**从ROM读指令，PC指针pAddr = 0x0000 0008，控制器根据机器码产生对应的控制信号。
 - 此时\$2寄存器的值为：2。



顶层仿真

- 观察结果：复位后第3个时钟周期，应该执行 `sw $2, 8($2)`（机器码:ac420008）
 - 周期时钟下降沿（150ns处）执行指令，写RAM、改变PC指针，执行后，如151ns处：
 - 机器码spo = ac420008
 - $\$2 \rightarrow (\$2 + 8)$ ，（之前 $\$2$ 的值：2），从仿真可知：Ram的数据线spo=2，地址线为aluRes = 0a；PC指针pAddr = 0x0000 000c，指向了下一个地址。



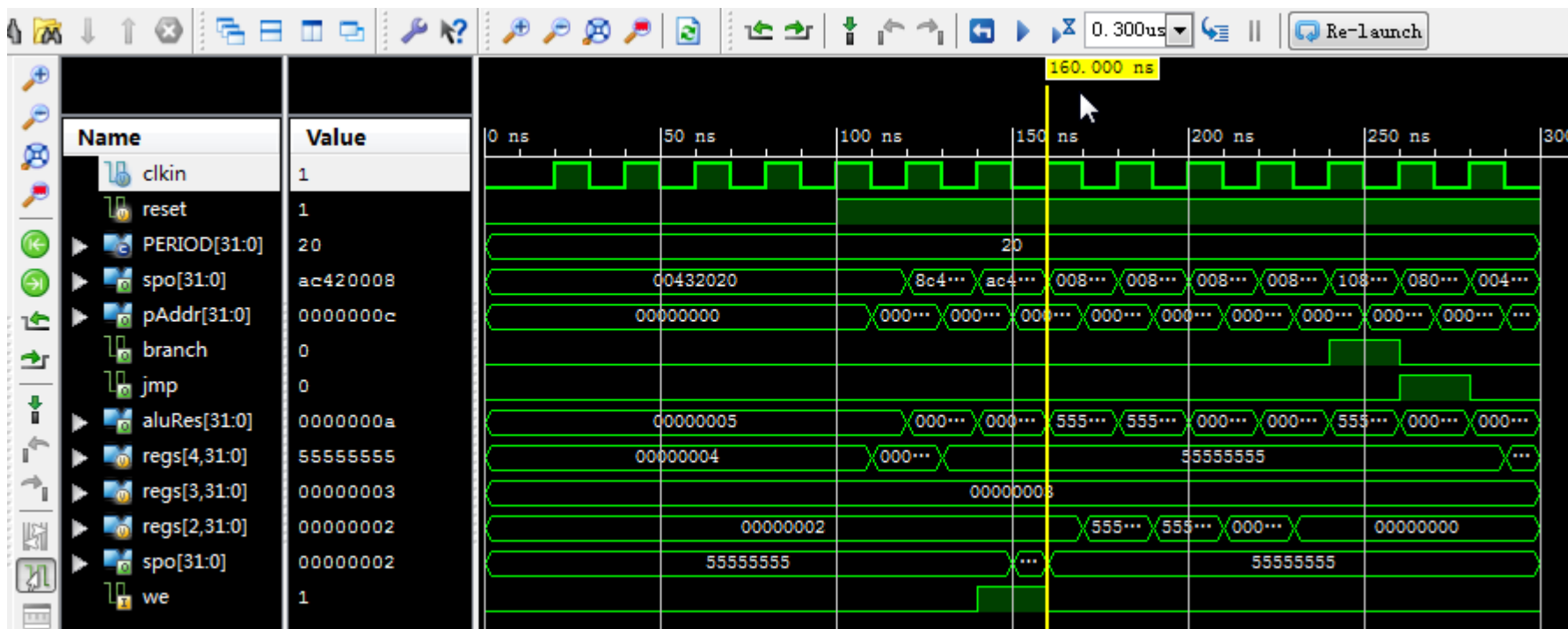
实际为地址8的字单元，因为ram硬件按照字读写：
.a(aluRes [7:2]),

RAM区08地址存储单元已经变为\$2中的内容2（需要保证仿真时没有后继指令修改过此存储单元），由此可见，lw指令被正确执行。



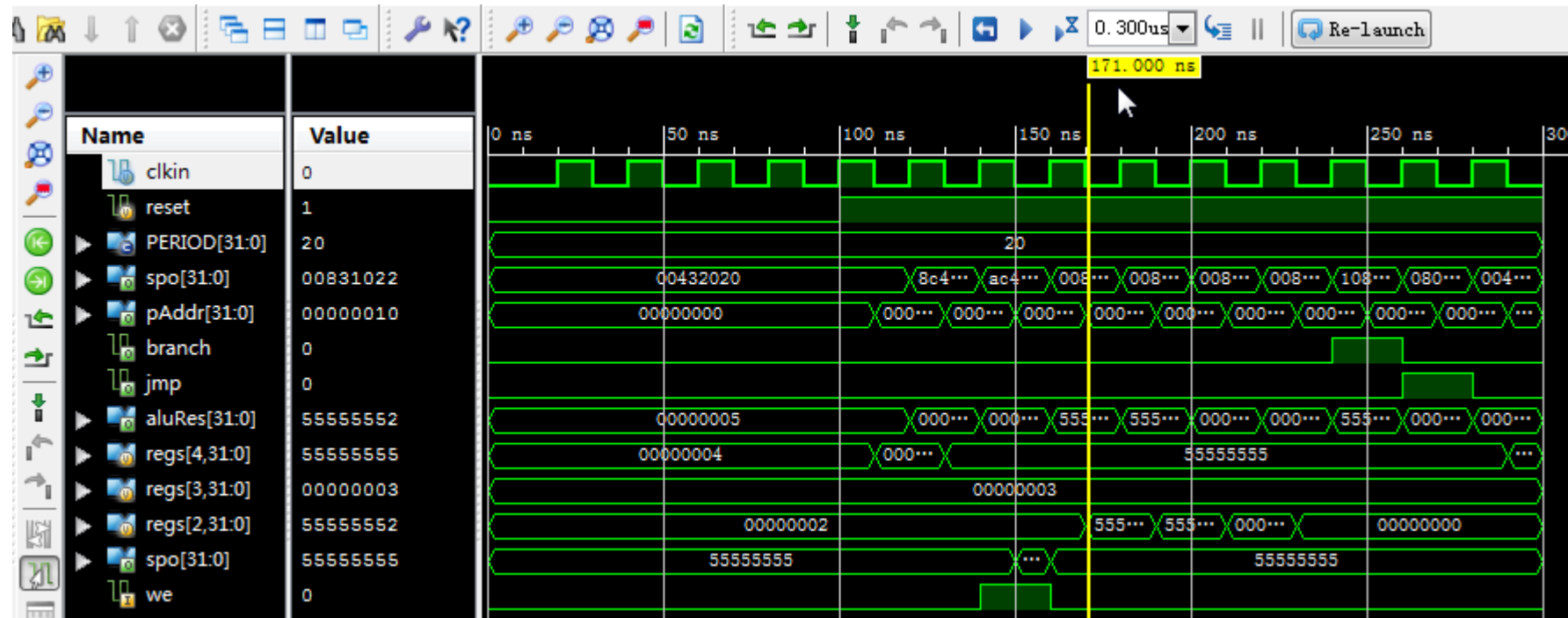
► 顶层仿真

- 观察结果：复位后第4个时钟周期，应该执行 sub \$2, \$4, \$3 （ 机器码:00831022 ）
 - 周期时钟上升沿（160ns处）从ROM读指令，PC指针pAddr = 0x0000 000c，控制器根据机器码产生对应的控制信号。
 - 此时\$2，\$3，\$4寄存器的值为：2，3，0x55555555。



► 顶层仿真

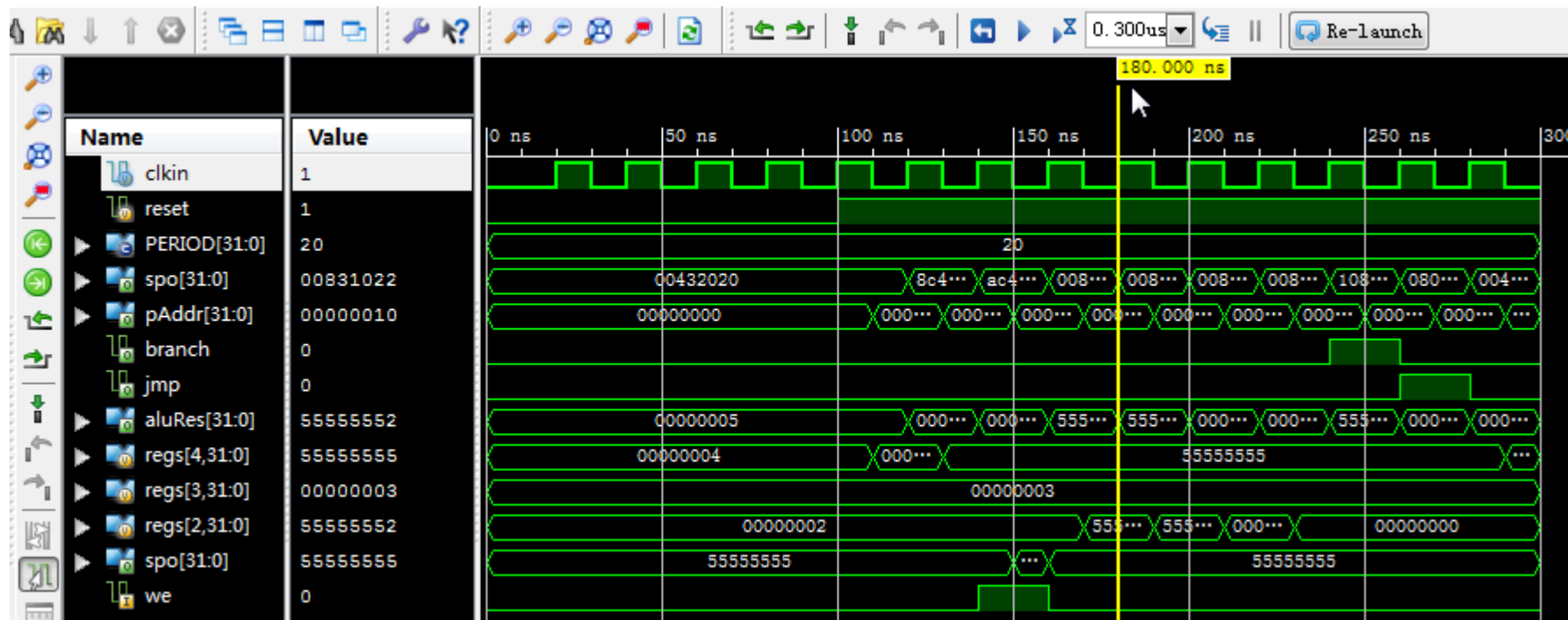
- 观察结果：复位后**第4个**时钟周期，应该执行 `sub $2, $4, $3`（机器码:00831022）
 - 周期时钟**下降沿**（**170ns处**）执行指令，写Reg、改变PC指针，执行后，如**171ns处**：
 - 机器码spo = 00831022
 - $\$4 - \$3 \rightarrow \$2 = 0x55555552$ ，（之前 $\$2, \$3, \$4$ 寄存器的值为：2, 3, $0x55555555$ ）；从仿真可知： $\text{reg}[2, 31:0] = 0x55555552$ ，PC指针pAddr = 0x0000 0010，指向了下一个地址。



由此可见，`sub` 指令被正确执行。

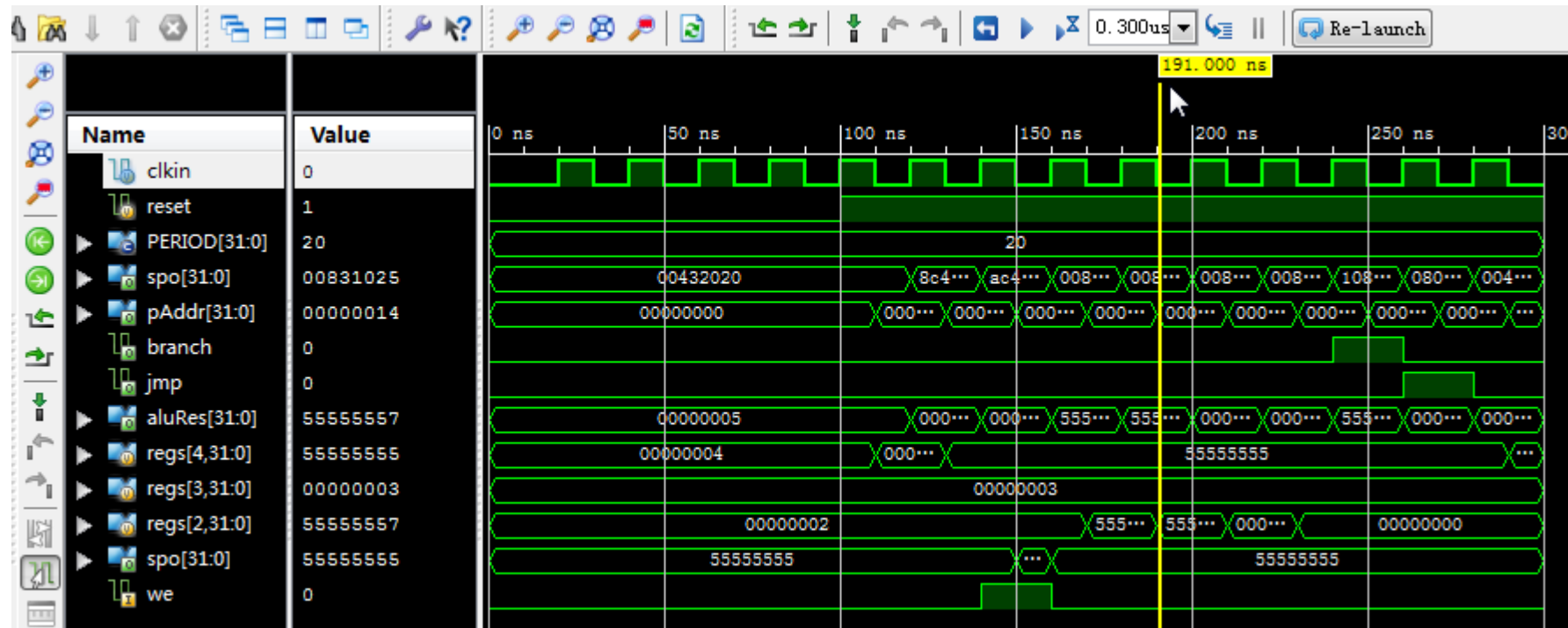
► 顶层仿真

- 观察结果：复位后**第5个**时钟周期，应该执行 or \$2, \$4, \$3 （机器码:00831025）
 - 周期时钟**上升沿（180ns处）**从ROM读指令，PC指针pAddr = 0x0000 0010，控制器根据机器码产生对应的控制信号。
 - 此时\$2，\$3，\$4寄存器的值为：0x55555552，3，0x55555555。



► 顶层仿真

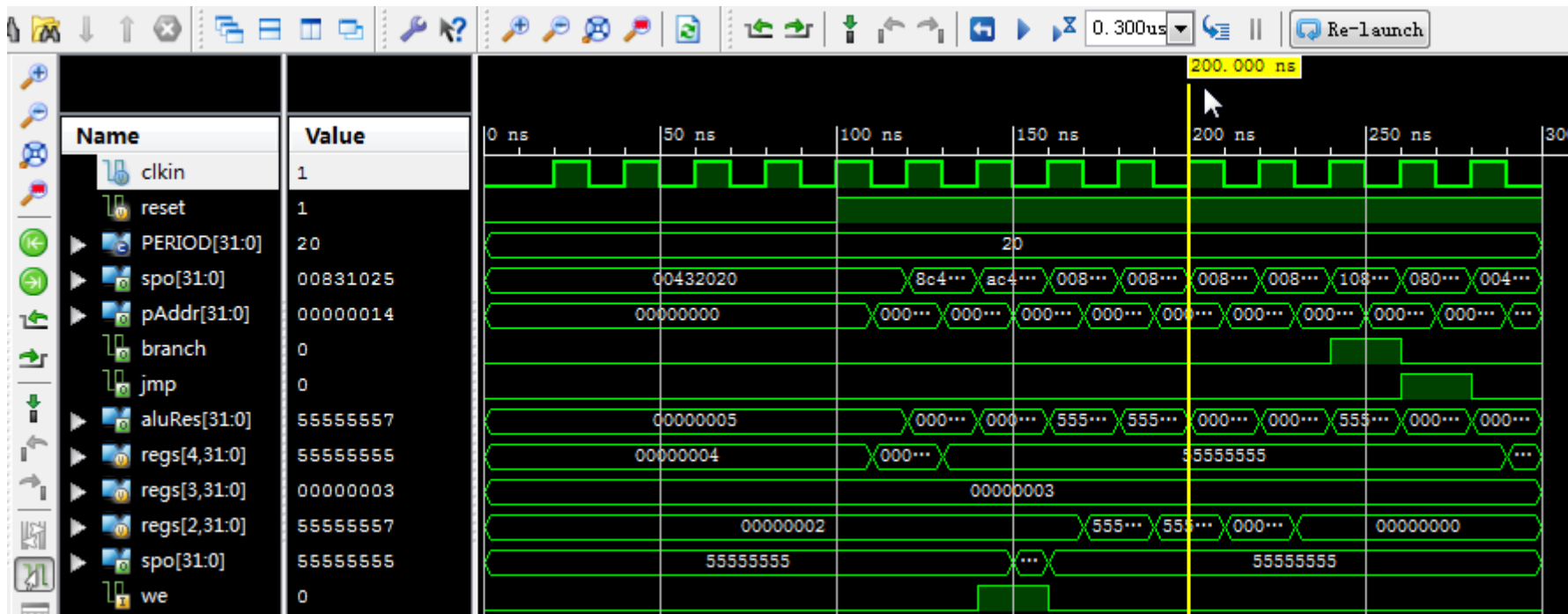
- 观察结果：复位后**第5个**时钟周期，应该执行 or \$2, \$4, \$3 （机器码:00831025）
 - 周期时钟**下降沿**（**190ns处**）执行指令，写Reg、改变PC指针，执行后，如**191ns**处：
 - 机器码spo = 00831025；\$4|S3 → \$2=0x55555557，（之前\$2，\$3，\$4寄存器的值为：0x55555552, 3, 0x55555555）；从仿真可知：reg[2,31:0]=0x55555557，PC指针pAddr = 0x0000 0014，指向了下一个地址。



由此可见，or 指令被正确执行。

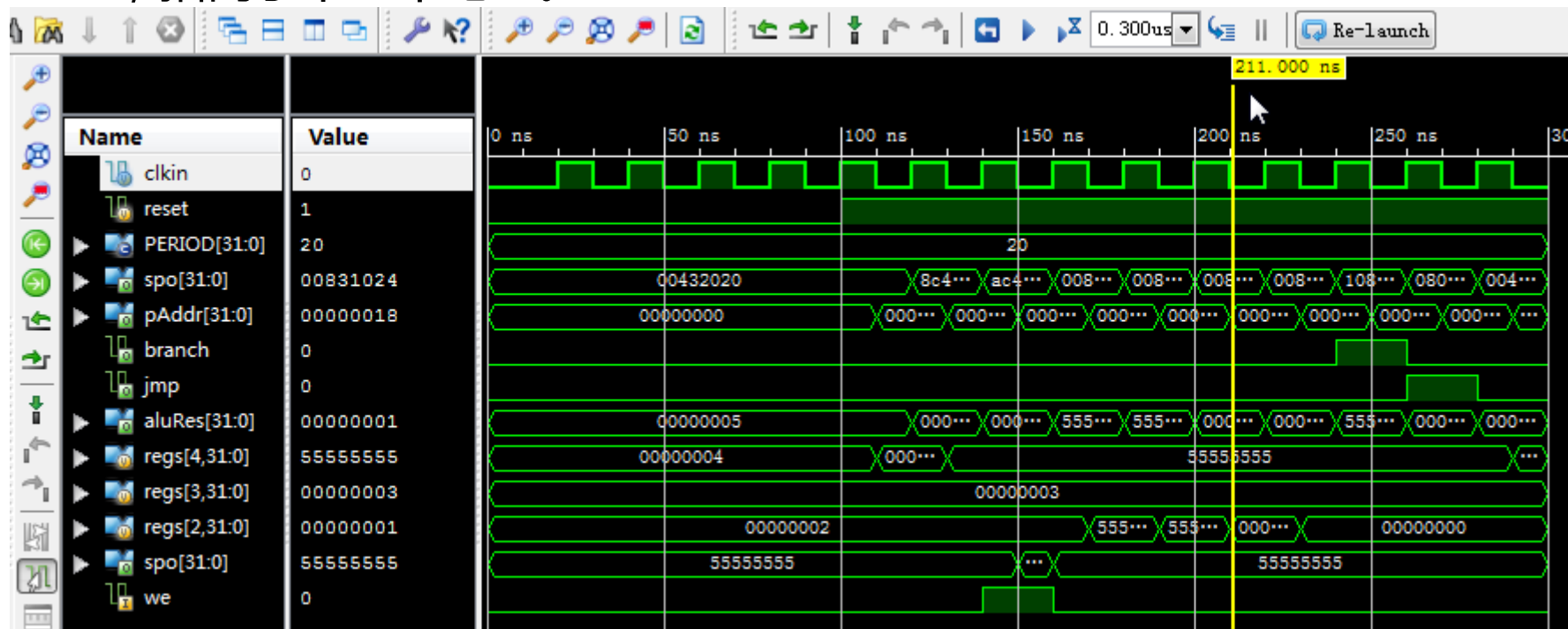
► 顶层仿真

- 观察结果：复位后**第6个**时钟周期，应该执行 and \$2, \$4, \$3 （ 机器码:00831024 ）
 - 周期时钟**上升沿（200ns处）**从ROM读指令，PC指针pAddr = 0x0000 0014，控制器根据机器码产生对应的控制信号。
 - 此时\$2，\$3，\$4寄存器的值为：0x55555557，3，0x55555555。



► 顶层仿真

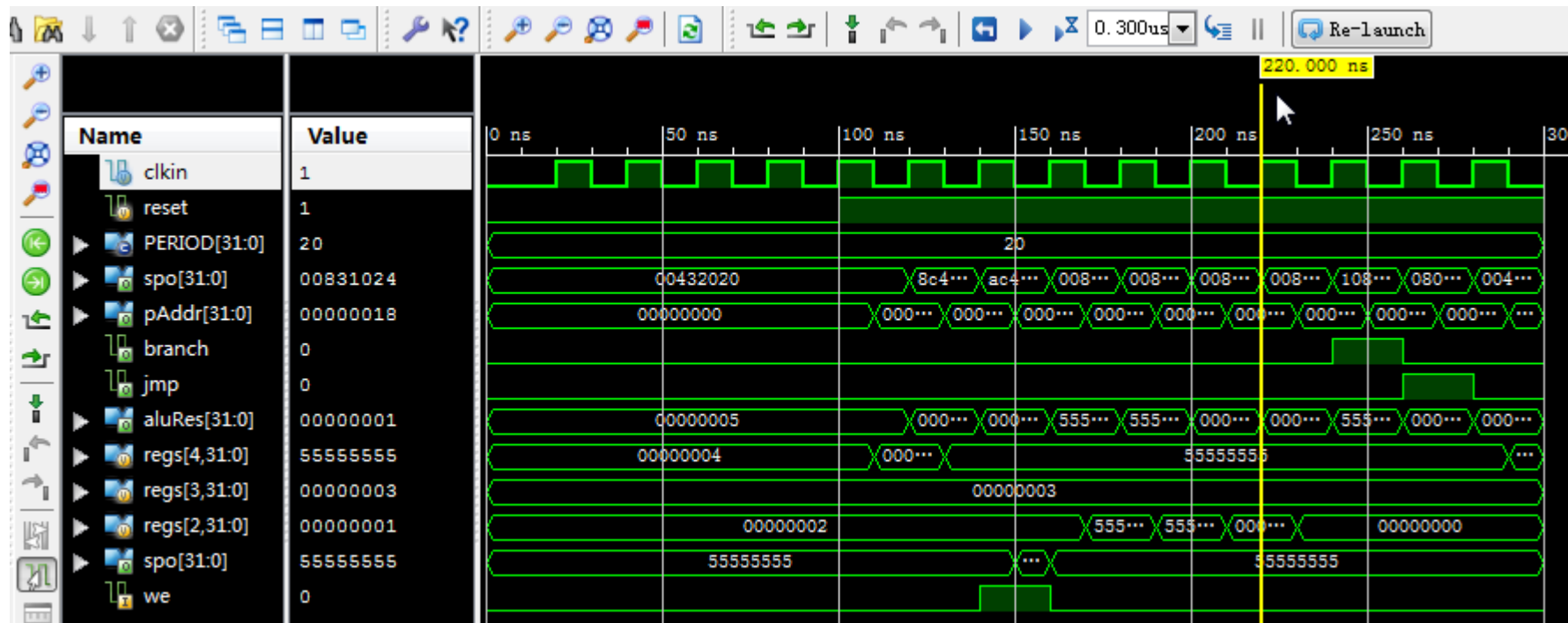
- 观察结果：复位后**第6个**时钟周期，应该执行 and \$2, \$4, \$3 （ 机器码:00831024 ）
 - 周期时钟**下降沿**（ **210ns处** ）执行指令，写Reg、改变PC指针，执行后，如**211ns**处：
 - 机器码spo = 00831024；\$4&\$3 → \$2=0x00000001，（ 之前\$2，\$3，\$4寄存器的值为：0x55555557, 3, 0x55555555 ）；从仿真可知：reg[2,31:0]=1，PC指针pAddr = 0x00000018，指向了下一个地址。



由此可见
，and 指
令被正确
执行。

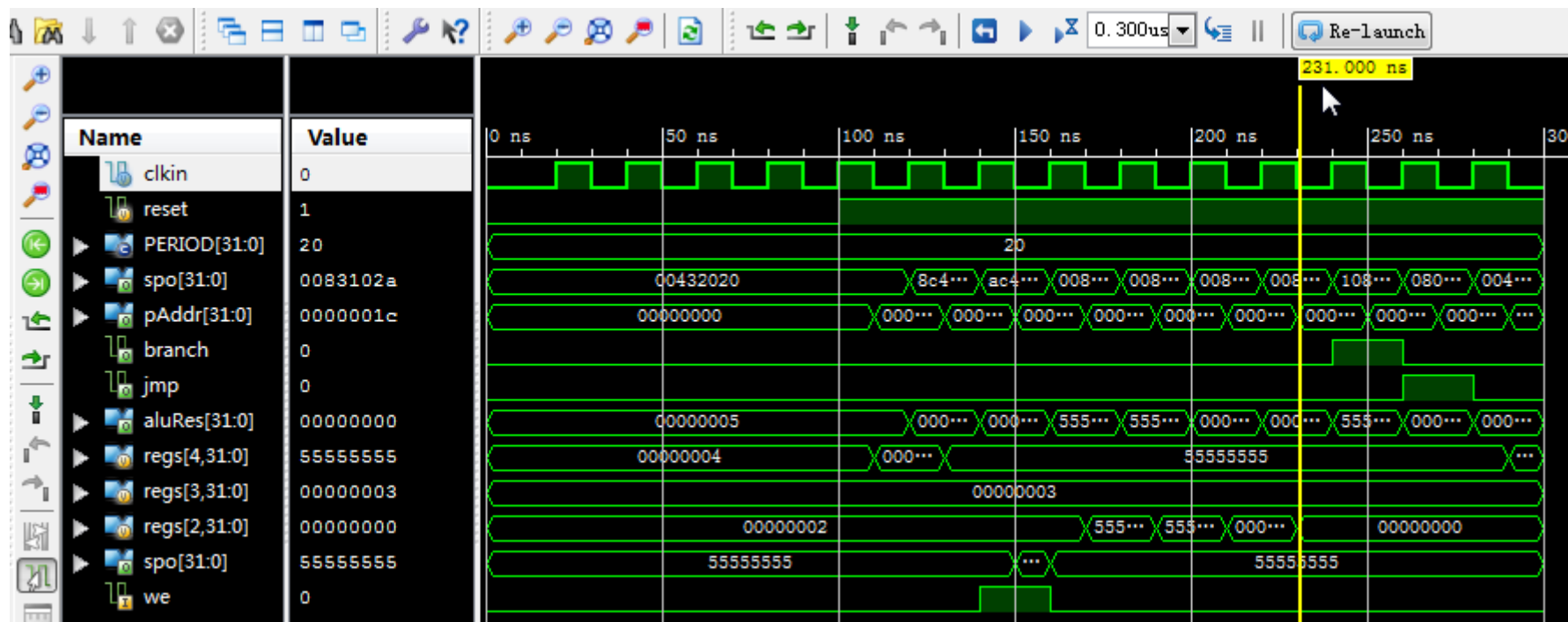
► 顶层仿真

- 观察结果：复位后**第7个**时钟周期，应该执行 `slt $2, $4, $3`（机器码:0083102a）
 - 周期时钟**上升沿（220ns处）**从ROM读指令，PC指针pAddr = 0x0000 0018，控制器根据机器码产生对应的控制信号。
 - 此时\$2，\$3，\$4寄存器的值为：1，3，0x55555555。



► 顶层仿真

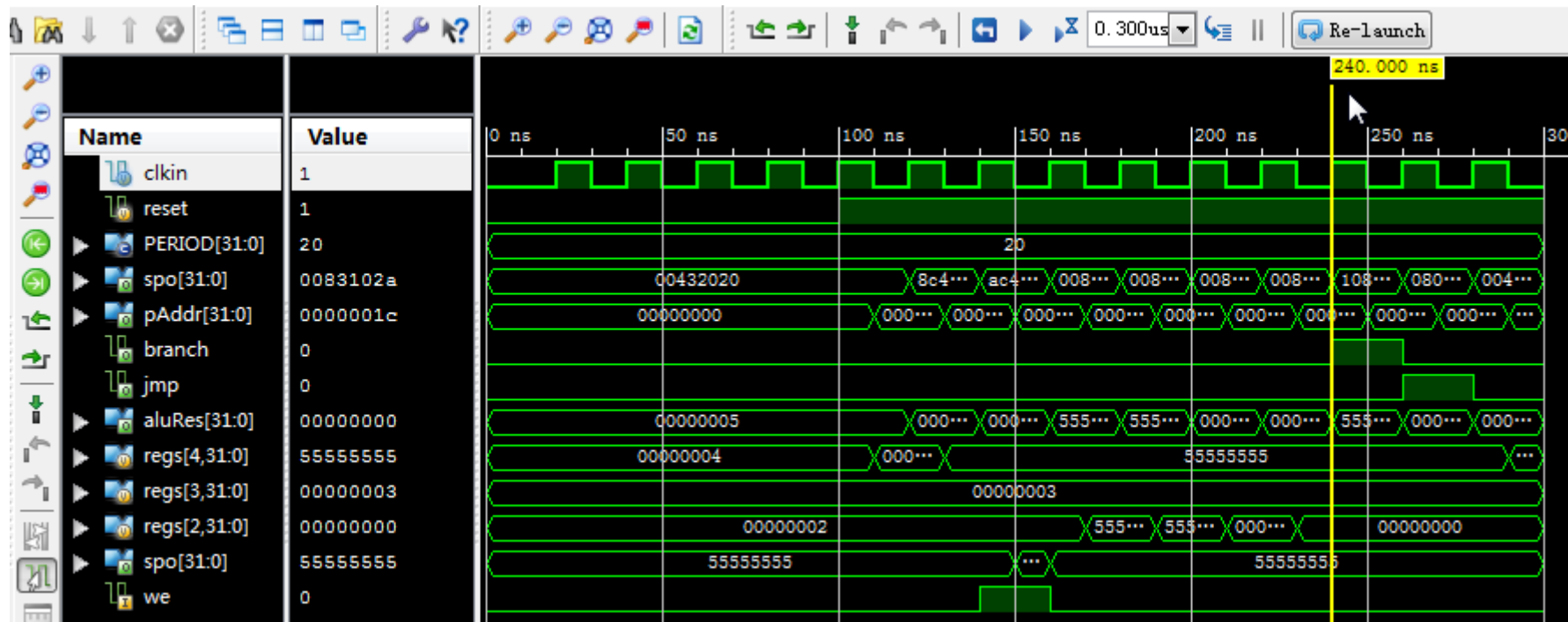
- 观察结果：复位后**第7个**时钟周期，应该执行 `slt $2, $4, $3` （机器码:0083102a）
 - 周期时钟**下降沿**（**230ns处**）执行指令，写Reg、改变PC指针，执行后，如**231ns处**：
 - 机器码spo = 0083102a； $\$4!<S3$ ，所以 $\$2=0$ ，（之前 $\$2$ ， $\$3$ ， $\$4$ 寄存器的值为：1, 3, 0x55555555）；从仿真可知： $\text{reg}[2,31:0]=0$ ，PC指针pAddr = 0x0000 001c，指向了下一个地址。



由此可见
，slt 指令
被正确执
行。

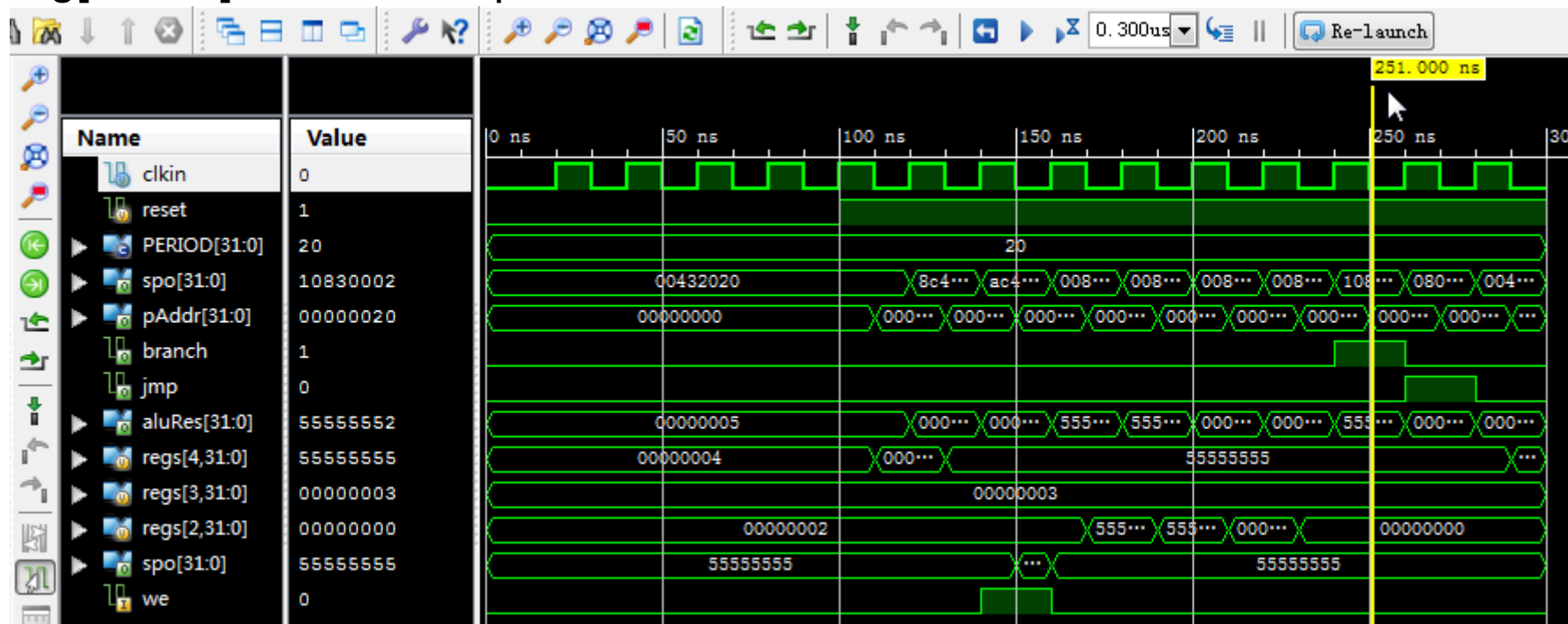
► 顶层仿真

- 观察结果：复位后**第8个**时钟周期，应该执行 beq \$4, \$3, exit (机器码:10830002)
 - 周期时钟**上升沿 (240ns处)**从ROM读指令，PC指针pAddr = 0x0000 001c，控制器根据机器码产生对应的控制信号。
 - 此时\$4，\$3寄存器的值为：0x55555555，3。



► 顶层仿真

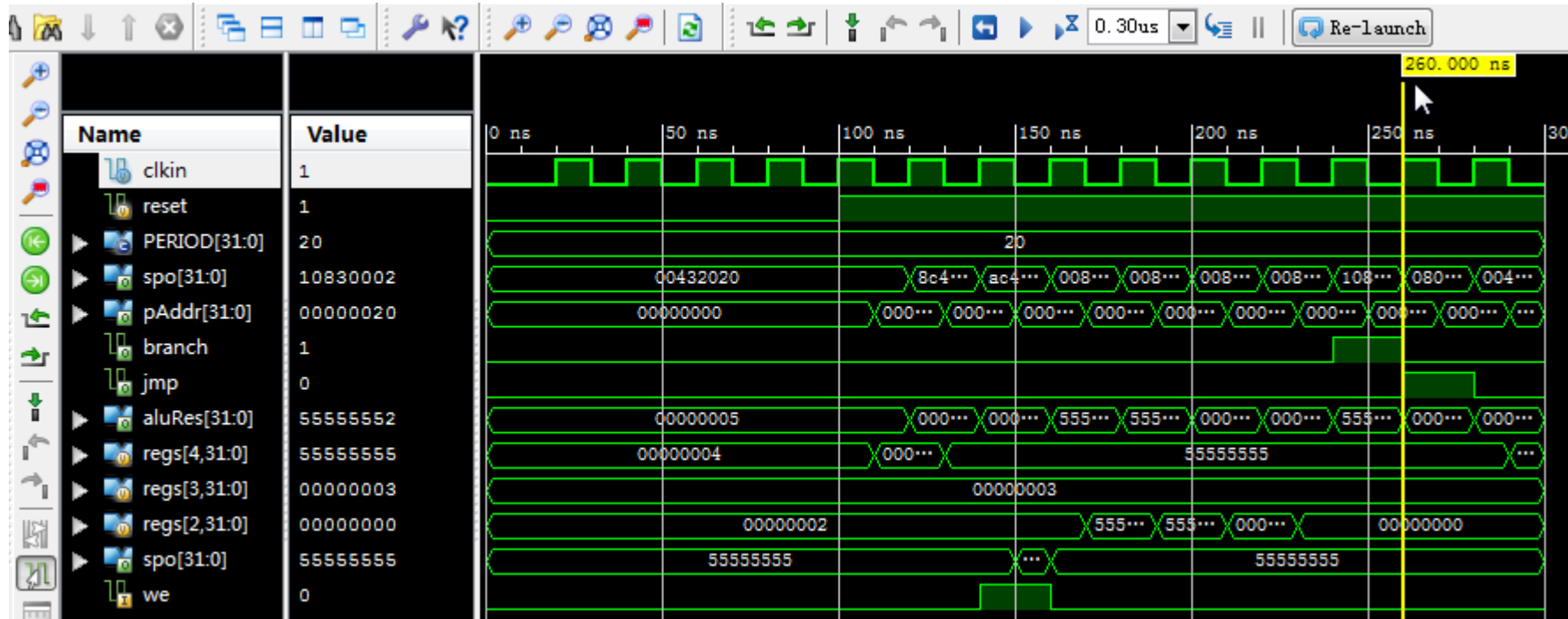
- 观察结果：复位后**第8个**时钟周期，应该执行 beq \$4, \$3, exit (机器码:10830002)
 - 周期时钟**下降沿** (**250ns处**) 执行指令，写Reg、改变PC指针，执行后，如**251ns**处：
 - 机器码spo = 10830002；
 - \$4!=S3，所以顺序执行，（之前\$4，\$3寄存器的值为：0x55555555，3）；从仿真可知：reg[2,31:0]=0，PC指针pAddr = 0x0000 001c + 4 = 0x0000 0020，顺序执行。



由此可见，beq指令顺序部分被正确执行。
如何仿真验证跳转部分？

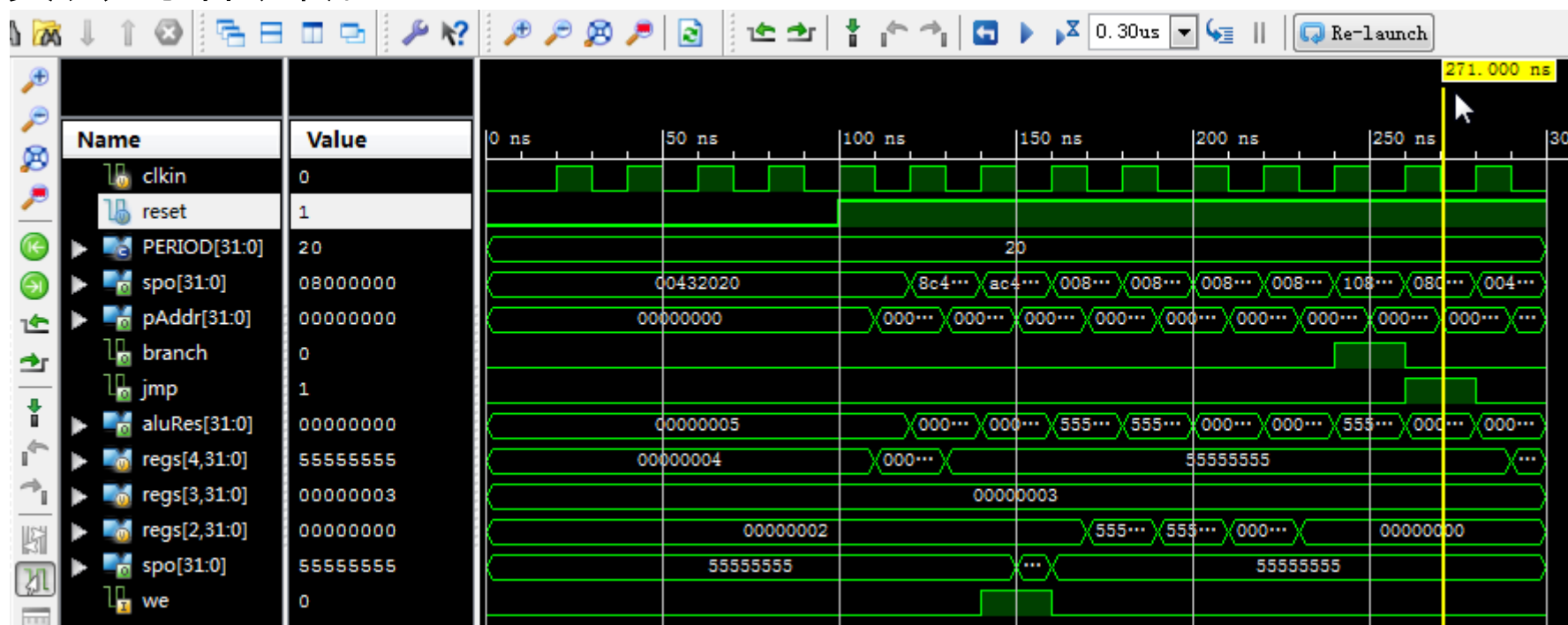
► 顶层仿真

- 观察结果：复位后**第9个**时钟周期，应该执行 j main （ 机器码:08000000 ）
 - 周期时钟**上升沿（260ns处）**从ROM读指令，PC指针pAddr = 0x0000 0020，控制器根据机器码产生对应的控制信号。



► 顶层仿真

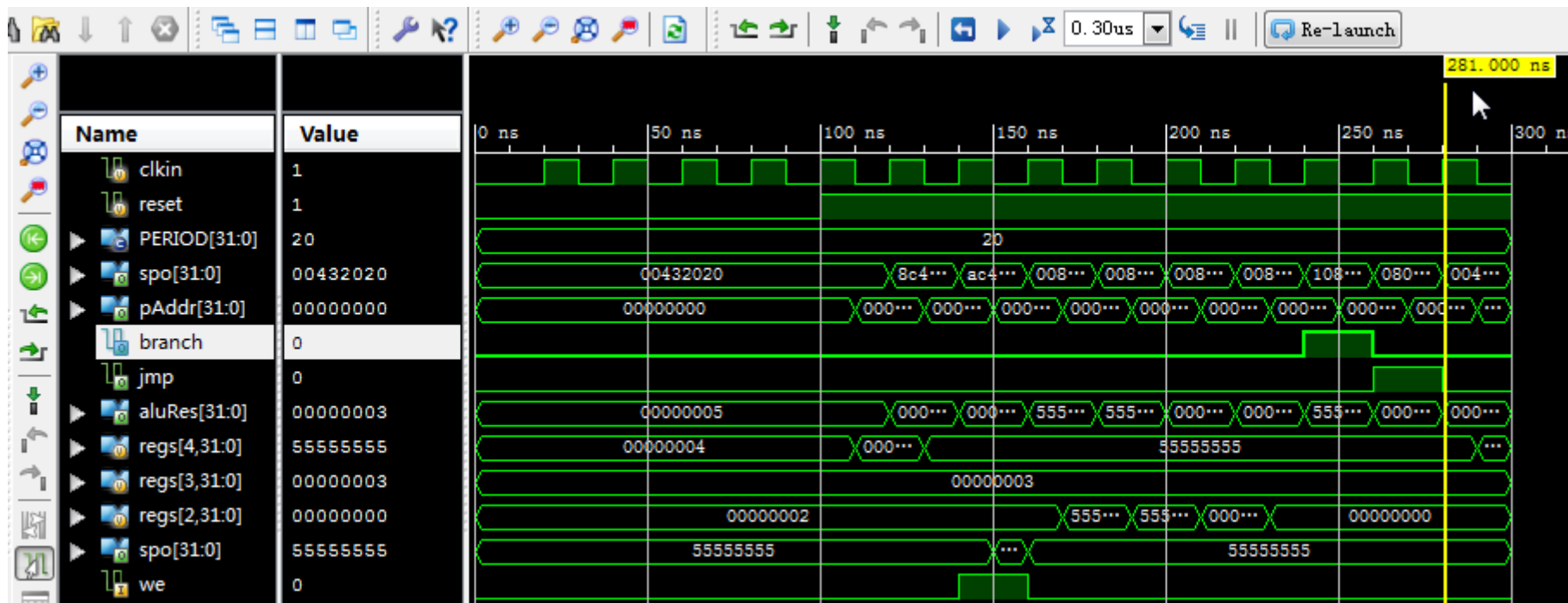
- 观察结果：复位后**第9个**时钟周期，应该执行 j main （ 机器码:08000000 ）
 - 周期时钟**下降沿**（ **270ns处** ）执行指令，写Reg、改变PC指针，执行后，如**271ns处**：
 - 机器码spo = 08000000；
 - 直接跳转到main，即跳到00000000地址处；从仿真可知：PC指针pAddr = 0x0000 0000，实现无条件跳转。



由此可见，j指令被正确执行。

► 顶层仿真

- 观察结果：复位后**第10个**时钟周期，由于之前的j main 指令，应该执行00000000处的第一条指令 add \$4, \$2, \$3 （ 机器码:00432020 ）
 - 周期时钟**上升沿（ 280ns处 ）**，从ROM读指令，此后，如281ns处，PC指针pAddr = 0x0000 0000，机器码spo = 00432020，表明已经跳转到main，重新执行程序段。



即程序中的代码会被被循环不断执行。至此，所有指令的功能仿真都已完成，仿真结果说明CPU功能正确。

► 顶层仿真

- 如果在前述仿真过程中发现某一指令功能不正确，需要查找原因、修改代码，重新进行仿真；
- 查找错误原因的方法
 - 观测相关信号时序
 - 设置断点
 -

Thanks

